

Test Resource Metrics

These are measures of effort, time-scale and staffing levels, and other cost items such as tool purchase, special test equipment, training etc. Test resource metrics are used for planning monitoring testing activities.

Test Coverage Metrics

These are among the most useful testing measures. They measure the amount of the product that has been tested at different levels of granularity. You need to identify the total set of features associated with aspect of a program/software module and measure the percentage of those features tested. In terms of "risk based approach to testing" discussed in the next section, this means prioritizing these features depending the importance of those to the customer. These features are often code features such as the number of logical paths or statements in a piece of software.

Test coverage statistics identify what percentage of the available paths or statements have been executed by a set of tests. You can set standards for test coverage metrics, such as a minimum 90 percent of branches and 100 percent of statements executed per module, which allows you to control the testing process. Test coverage of code is usually part of a white box testing regime for unit testing. However, test coverage measures can reference module-calling paths as part of integration testing or reference requirements as part of functional system testing.

In general, low test coverage values for a module are an indication of incomplete/poor testing of a specific component. Test coverage values should be checked when modules with unusually high or low defect rates are identified.

Test Plan Metrics

These indicate the amount of testing to be performed and the effectiveness of test plans. Examples of such metrics are the number of test cases planned and run vis-à-vis the number of unplanned test cases run. Like testing resource metrics, testing plan metrics are used for planning and monitoring testing activities.

10.11.3 Test Result Metrics

These are usually based on defect counts. Such counts can be related to the test item and the test activity in terms of error rates (e.g. defects per 100 lines of code and defects found per 100 testing hours respectively).

Defect counts can also be made in terms of various defect categories. A useful defect classification scheme would identify the phase and/or activity during which the error was introduced, the phase/activity during which the error was detected, and the type and severity of the error. This would eventually help move towards *orthogonal defect classification* famously known as ODC (readers are urged to use the web for ODC as the discussion on this topic is beyond the scope of this chapter).

Defect rates are often used to identify the pass and fail criteria for test item (refer Table 4—Test Results in section 10.7). For example, a module may be rejected if its defect rate exceeds some predetermined limit (organizations find out the upper and lower limits based on their capability baseline which in turn is established using past performance data of their software projects.) System testing might be required to continue until the defect rate per test hour drops below some predetermined level. Defect classification is used to investigate process efficiency and to identify potential weakness in the process.

10.12 Risk-Based Testing

Basically, "Risk-based Testing" is an approach to developing a test strategy. We'll mention about the famously known *White Box Testing* and *Black Box Testing* strategies at the end of this section, along with the new term "Gray Box Testing".

Test strategies are adopted in recognition of the fact that there are a number of strategic risks associated with the development and installation of a computer-based system can be [1]:

- Incorrect results will be produced
- Unauthorized transactions will be accepted by the system
- Data integrity will be lost
- Processing cannot be reconstructed
- Continuity of processing will be lost
- Service provided to the user will degrade to an unacceptable level
- Security of the system will be compromised
- Processing will not comply with organizational policy of statutory and regulatory/legal requirements
- Results of the system will be unreliable
- System will be difficult to use
- Programs will be un-maintainable
- System will not portable to other hardware and software
- System will not be able to interconnect with other computer systems
- Performance level will be unacceptable
- System will be difficult to operate

Each of these risks can affect the proper functioning of a computer-based application. If any of any of these risks materialize, the result may be substantial loss to the organization using the system. In the section of this chapter testing techniques –we discussed how most of the above risks are addressed while testing “non-functional requirements”.

The Test Plan should be in line with the organization’s testing strategy. In forming a test strategy, indemnification of the “risk factors” is an important step in determining the objective of testing. This is the basis of “risk-based approach to testing”. We have enumerated some of the *test factors*. Detailed discussion is not within the scope of this chapter. Interested readers can refer [1] in the reference section: *Correctness, File Integrity, Authorization, Audit Trail, Continuity of Processing, Service Levels, Access Control, Compliance, Reliability, Ease of Use, Maintainability, Ease of use, Portability, Performance, Coupling, Ease of operations*.

Developing a Test Strategy involves carrying out the following steps:

1. Identify and rank test factors
2. Identify the system development phases
3. Identify the business risks associated with the system under development
4. Place the risks in the matrix

This results in Test Factor/Test Phase Matrix as shown in Figure 5 below. This matrix is an important input to developing a Test Strategy. A detailed testing plan is required to ensure that the *test factors* have been adequately addressed. While the factors are common to the entire testing life cycle, the concerns vary according to the phase of the life cycle.

TEST PHASE TEST FACTORS	Requirements	Design	Build	Dynamic Test	Integrate	Maintain

Figure 5: Test Factor/Test Phase Matrix

Earlier in this section, we mentioned that there are three important terms—*White Box Testing* (also known as “Glass Box Testing”), *Black Box Testing* and *Gray Box Testing* (a relatively new term). These are basically the commonly known “testing strategies”—a brief description is provided below.

White Box Testing (Also called “Glass Box Testing”): This is a totally logic driven testing and permits one to examine the internal structure of a software code/program. In using this strategy, the tester derives test data from an examination of program’s logic (and often, unfortunately at the neglect of the specification!) White Box Testing considers close examination of procedural details. Executions of logical paths are considered by providing test cases that exercise specific set of conditions and/or loops. The status of program may be examined at various points in execution to determine if the expected or asserted status (for example some compilers provide “tracing” utilities to check if the control points are being reached) corresponds to the actual results.

Black Box Testing: We had said earlier exhaustive testing is virtually impossible i.e. you cannot find *all* errors even for the most trivial programs. It is highly impractical too. Black Box Testing is used as one way to examine this issue. This is completely data-driven or input/output driven. In using this strategy, the tester is not at all concerned with the internal behaviour/structure/working of the software code. Rather, the tester is only interested finding the circumstances in which the program/software code does not behave according to its specification. Thus, Black box testing refers to tests that are conducted at the software interface. Black Box tests are generally used to demonstrate that software functions are operational. For a very exhaustive treatment of this topic of Black Box testing, reader may like to see Ref [7].

Gray Box Testing: as said earlier, this is a relatively new term coming up due to the rise of web-applications. This is mixture of White Box Testing and Black Box Testing. With Gray Box Testing, you straddle between the two. Your strategy will be to test the software as black box but you supplement the work by taking a peek (not a full look inside as in case of White Box testing) at what makes the software work. Web pages lend themselves nicely to Gray Box Testing. Web pages are created mostly using HTML. This is suitable for using Gray Box testing Strategy because HTML is not a full-fledged programming language; it is a mark-up language.

Having discussed *White Box Testing*, *Black Box Testing* and *Gray Box Testing*, we now turn our attention to another important term—*Regression Testing*. As you’ll see in Chapter 5, while discussing *software maintenance metrics*, we have made a reference to this term. In this section, we add a discussion on regression testing to bring in completeness to the topic of testing addressed by this chapter.

What is Regression Testing?

Regression testing has a specific purpose; it is to ensure that the *Application Under Test* (AUT) still functions correctly even after carrying out the required modification or extension of the system (such as enhancements or upgrades). Typically, the modifications or extensions will be tested to make sure that they meet their requirements, after which a series of tests will be run to confirm that the existing functionality still meets its requirements.

Strictly speaking, regression testing is not a testing phase and as such, the V-Model for testing does not mention it. Rather, regression testing is a *testing technique* that can be used with any of the testing phases described in the previous sections. Regression testing can also be employed following changes to the operating environment of the AUT; such as the installation of the new version of the operating system, changes to the hardware platform, or a particular event external to the AUT. A very well known example of this is what happened when the world entered the famous year 2000! *Millennium Testing* can be considered to have been a special case of Regression Testing. During a typical millennium test, the AUT would not have been altered, but it would still be necessary to ensure that the previously designed applications performed correctly following a particular event (in this case the year 2000 date change).

Typically, Regression Testing is employed in the context of System and Acceptance Testing. Generally, Testing experts are of the opinion that Regression Testing *should* use the Black Box testing strategy to test the high-level requirements of the AUT without considering the implementation details of the system.

Regression Testing is a particularly appropriate candidate for support by means of an automated testing tool, especially if frequent new builds and releases of the AUT are anticipated. The advantages of using test automation during regression testing are:

- The ability to run the regression tests "unattended" i.e. without the need for the testing staff to be present
- The ability to rerun all the automated test scripts or selected subset of scripts against which the new build or release as required to provide maximum confidence in the software
- Establishing a high degree of confidence that the modifications or extensions to the system have not impacted unduly on its functionality

10.13 Test Automation and Test Tool Selection

In this section, we are going to have a general discussion on how to go about selecting a test automation tool. Automation is one of the factors promoting higher productivity. It is a fact that often testing is perceived as a boring job and this feeling of "boredom" (whether perceived wrongly or not) can leave to a loss of productivity. The other problem is that in an attempt to do more with less, organizations want to test their software adequately, but within a minimum schedule! Manual testing is labour-intensive and error-prone. It is impractical given the complexity of design architecture of today's software solutions. The automation of test activities provides its greatest values in instances where *test scripts* are repeated or where test scripts sub-routines are created and then invoked repeatedly by a number of test scripts. Thus the payoff from automated testing is greatest when the testing tasks are repetitious.

To understand the context of automated testing, it is necessary to understand the kinds of test, which are typically performed during the various application development life-cycle stages. Within a client-server or Web environment, the target system spans more than just a software application. Indeed, it may perform across multiple platforms, involve multiple layers of supporting applications, involve interfaces with a host of commercial off-the-shelf (COTS) products, utilize one or more different types of databases, and involve both front-end and back-end processing. Tests within this environment can include Functional Requirements Testing, Server Performance Testing, User Interface Testing, Unit Testing, Integration Testing, Program Module Complexity Analysis, Program Code Coverage Testing, System Load Performance Testing, Boundary Testing, Security Testing, Memory Leak Testing, and many more types of checks. In Figure 3, the overall Testing Life Cycle was depicted. Figure 6 details out the Test Tool selection aspect.

The decision to automate test represents an important decision during the testing activity.

Test Tool Acquisition is the next important activity. The process of *Introducing Automated Testing* to a new project comes next. Refer Figure 3—tool selected forms an important part during the preparation of the Test Plan. With this background, now let us discuss *Automated Test Tool Selection Process* (Figure 6). The person involved in this activity needs to develop a *test tool proposal* for management—the proposal outlines the test tool requirement and the justification for the tool. Test tool proposal development and its acceptance by management are intended to secure management's commitment for the resources needed to properly implement the test tool and support the automated testing process. To conclude this section, Table 7 presents a summary of Test Life-Cycle Tools.

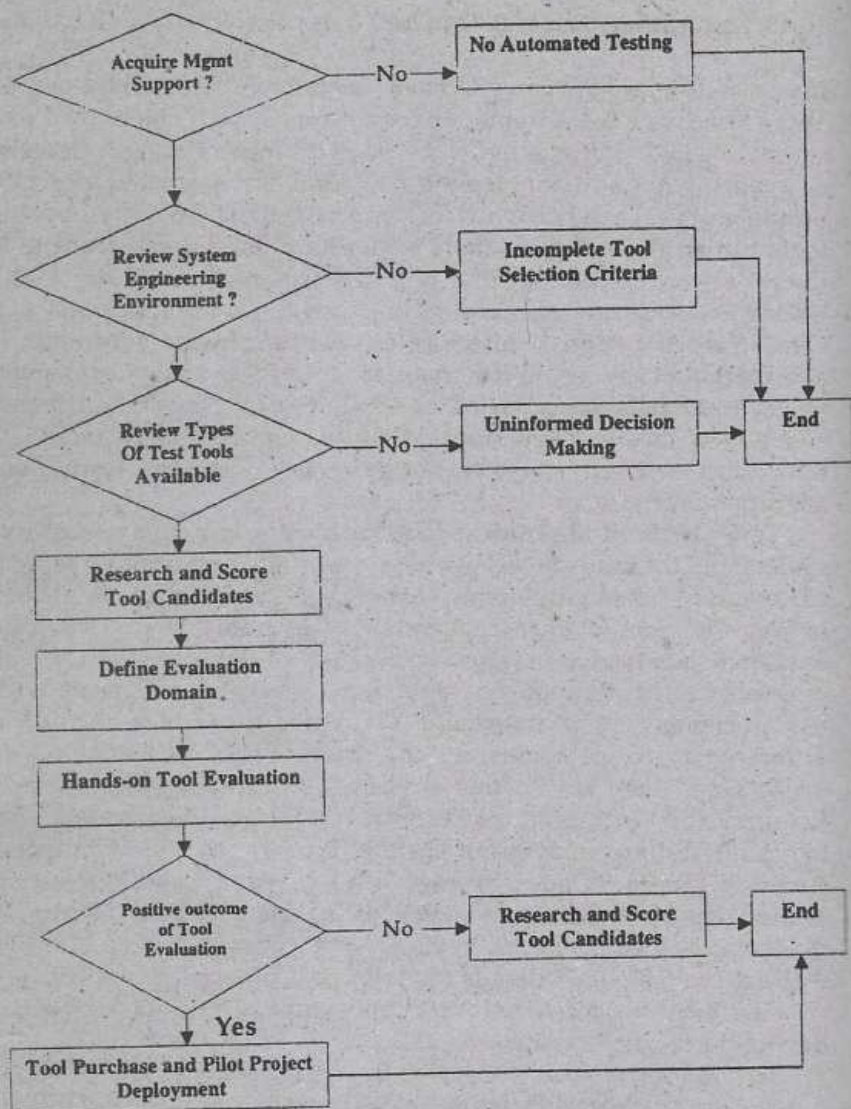


Figure 6: Automated Test Tool Selection Process

Table 7: Test Life-Cycle Tools

<i>Life Cycle Phase</i>	<i>Type of Tool</i>	<i>Tool Description</i>
Business Analysis Phase	• Business Modeling Tool	Allow for recording definitions of user needs and automating the rapid construction of flexible, graphical, client-server applications
	• Configuration Management Tool	Allow for base-lining of important data repositories
	• Defect Tracking Tools	Manage system life-cycle defects
	• Technical Review Management	Facilitates communication and automates the technical review/inspection process
	• Documentation Generators	Automate Document generation
Requirements Definition Phase	• Requirements management tools	Manage and organize requirements; allow for test procedure design; allow for test progress reporting
	• Requirements verifiers	Verify syntax, semantics, and testability
	• Use case generators	Allow for creation of use cases
Analysis and Design Phase	• Database Design Tools	Provides a solution for developing second-generation enterprise client-server systems
	• Application Design Tools	Define software architecture; allow for object oriented analysis, modeling, design, and construction
	• Structure charts, flowcharts, and sequence diagrams	Help manage process
	• Test procedure generators	Generate test procedures from requirements or design of data and object models or cause-effect diagrams
	• Syntax checkers/debuggers	Allow for syntax checking and debugging and debugging capability; usually come with built-in programming language compilers
Programming Phase	• Memory leak and runtime error detection tools	Detect runtime errors and memory leaks
	• Source code testing tools	Verify maintainability, portability, complexity, cyclomatic complexity, and standards compliance
	• Static and dynamic analyzers	Depict quality and structure of code

Life Cycle Phase	Type of Tool	Tool Description
Metrics Tools	• Various code implementation tools	Depending on the application, support code generation, among other things
	• Unit test tools	Automates the unit testing process
	• Code (test) coverage analyzers	Identify untested code and support dynamic testing
	• Metrics reporters	Read source code and display metrics information
Testing Phase	• Usability measurements	Provide user profiling, task analysis, prototyping, and user walkthroughs
	• Test management tools	Allow for test management
	• Network testing tools	Allow for monitoring, measuring, testing and diagnosis of performance across the entire network
	• GUI testing tools (capture/playback)	Allow for automated GUI tests; capture/playback tools record user interactions with on-line systems, so they may be replayed automatically
	• Non-GUI test drivers	Allow for automated execution of tests for products without GUIs
	• Load/performance testing tools	Allow for load/performance and stress testing
	• Environment testing tools	Testing tools for various testing environments

10.14 Extreme Testing

This is relatively new concept and becoming quite popular. Its roots are in the fact that requirements keep changing and that the delivery life cycles are shorter especially in the web world. Kent Beck is considered to be the father of "Extreme Programming" concept which has given rise to this new philosophy to testing called: *Extreme Testing*". The basic idea is: testing the system on the fly! The analogy given below by Kent Beck will help understand the philosophy in the Extreme Testing approach.

Kent Beck compares programming to driving a car. If you are driving a car, you don't simply point in the general direction of your destination and hold down the gas until you get there. Good, safe driving consists of making continual corrections until you reach your destination. If the road turns, you turn the steering wheel. If traffic is slowed to a crawl, you ease up on the accelerator. Extreme Testing takes the same approach from a development point of view. It is believed that change is not "bad," it's just part of the journey, and you cope with

it continually along the way. (As said above, extreme testing approach is in realization of the fact that "change" is the only reality about requirements!)

One of the basic principles of extreme programming is unit testing. In fact, extreme programming suggests that even before developer start writing code they should first design an automated unit test that will fail unless the coding task is implemented successfully. Once the unit test is created, the developer writes code until the unit test stops failing, at which point the task should be successfully finished.

Extreme Programming/Extreme Testing: A Manner of Thinking

Every good tester knows that unit testing is important. So why is unit testing considered "extreme programming?" If you think of extreme as meaning "risky" or "unproven" then no, unit testing is not extreme. However, if you think of extreme as meaning "nobody else does it" then yes, unit testing is extreme. If every developer implemented unit testing then it could no longer be called extreme. In fact, programmers who didn't write unit tests would be considered extreme!

A word of caution: Neither *Extreme programming* nor *Extreme Testing* claim to have the solution to every problem. You may be familiar with the phrase, "Our product variables are cost, time, and quality—choose any two." If you can decide to keep costs low and still deliver a product in a short time, quality will naturally suffer. Or if you choose to have high quality in a short delivery time, the development costs will skyrocket. However, as a philosophy, Extreme Testing is a good concept to keep the morale of the testing high throughout the project even in the face of continuously changing requirements.

10.15 Test Process Improvement Framework

We'll now discuss about "test process improvement" in the spirit of "continuous improvement"—the cornerstone of famous SW CMM (Capability Maturity Model for Software: details available at <http://www.sei.cmu.edu/>). For this discussion we'll use the *Test Maturity Model* (TMM). The TMM is a testing maturity that was developed by the Illinois Institute of Technology; it contains a set of maturity levels through which an organization can progress toward greater test process maturity. As shown in Figure 7, this model lists a set of recommended practices at each level of maturity above Level 1. The purpose of TMM is to promote greater professionalism in software testing, similar to the intention of the CMM for software that was developed by the Software Engineering Institute (SEI) at Carnegie Mellon University.

Interestingly, the TMM was developed as a complement to the CMM. It was envisioned that organizations interested in assessing and

improving their testing capabilities would likely be involved in general software process improvement. To have directly corresponding levels in both maturity models (SW CMM and TMM) would mean to have these two process improvement drives in parallel. This parallelism is not entirely present, however, because both the CMM and the TMM level structures are based on the individual historical maturity growth patterns of the processes they represent. The testing process is a subset of the overall software development process; therefore, its maturity growth needs support from the key process areas (KPAs) associated with the general process growth. For this reason, any organization that wishes to improve its testing process throughout implementation of the TMM (and the Testing Life-cycle) should first commit to improving its overall software development process by applying the CMM guidelines.

Research shows that an organization striving to reach a particular level of the TMM must be at least at the same level of the CMM. In many cases, a given TMM level needs specific support from KPAs in the corresponding CMM level and the CMM level beneath it. The KPAs should be addressed either prior to or in parallel with the TMM maturity goals.

The TMM model adapts well to automated software testing, because effective software verification and validation programs grow out of development programs that are well planned, executed, managed, and monitored. A good software test program cannot stand alone; it must be an integral part of the software development process. In Table 7, we have depicted the TMM levels together with corresponding automated software testing levels 1 through 5 in the second column as a part of TMM.

The test team must determine, based on the company's environment, the TMM maturity level that best fits the organization and the applicable software application or products. The level of testing should be proportional to complexity of design. And in general, the testing effort should not be more than the development effort. For readers interested in more details on the Test Maturity Model, we urge them to refer the information available on the internet for additional details on Test. One recommended site is: www.stsc.hill.af.mil/crosstalk/1998/nov/burnsetin.asp (It has an article by Ilene Burnstein, Ariya Homiyen, Robert Grom, C.R. Carlson of Illinois Institute of Technology)

Table 8 ahead depicts the levels 1 through 5 of the TMM in the first columns, along with corresponding automated software testing levels 1 through 5 in the second column. Column 2 addresses test maturity as it pertains to automated software testing. (Refer Figure 7 for Testing Maturity Model).

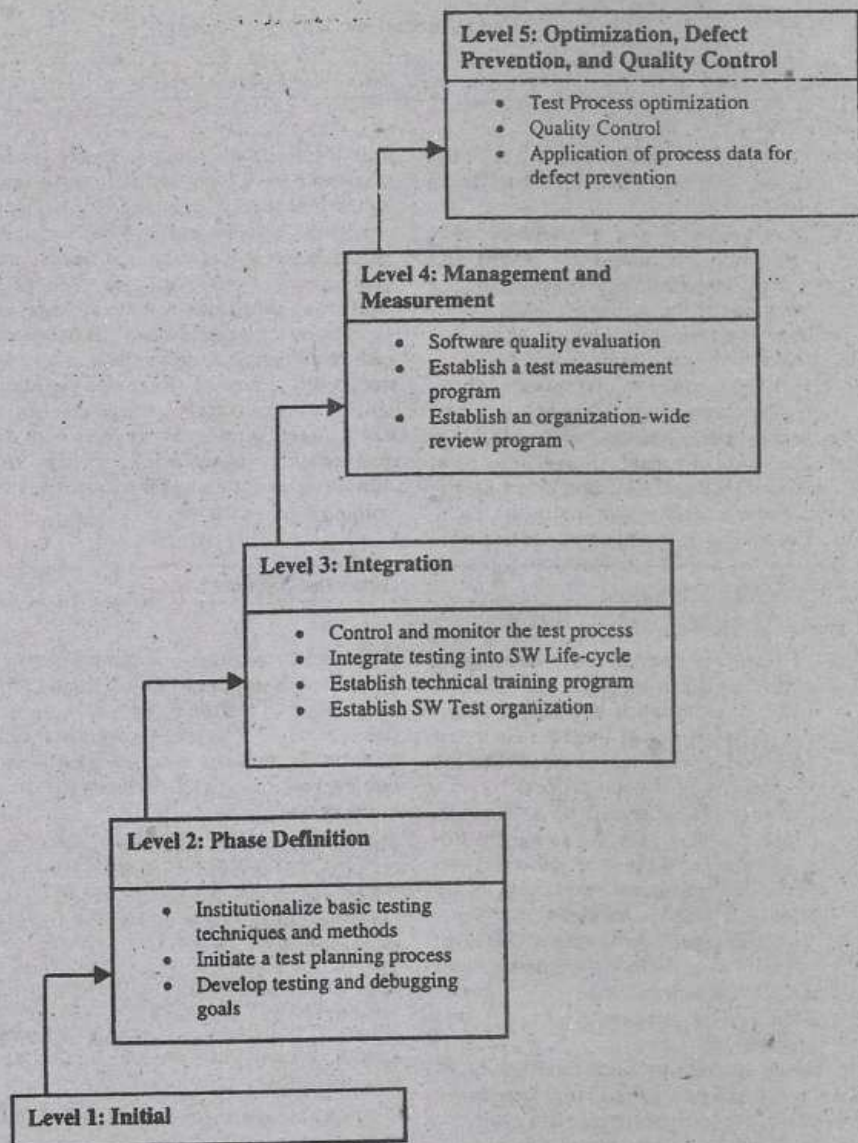


Figure 7: The 5-level Structure of the TMM

Table 8: TMM and Automated Software Testing

TMM Level 1	Automated Software Level 1
INITIAL	
• Testing is a chaotic process; it is ill-defined and not distinguished from debugging. • Tests are developed in an ad-hoc way after coding is completed. Testing and debugging are interleaved to get the bugs out of the software. • The objective of testing is to show that the software works. • Software products are released without quality assurance. Resources, tools, and properly trained staff are lacking. • This type of organization would be at level 1 of the CMM developed by the Software Engineering Institute • There are no maturity goals at this level	This level of automated software testing is referred to as "<i>accidental automation</i>". At the first level, automated testing is not done at all or only on an ad-hoc basis. An automated test tool might be used on an experimental basis. With a capture/playback tool, automated test scripts are reported and played back with only tool-generated scripts being used. Scripts are not modified for re-usability or maintainability. No automated script design or development standards are followed. Resulting scripts are not reusable and difficult to maintain and must be repeated with each software build.
TMM Level 2	Automated Software Level 2
PHASE DEFINITION	
• Testing is separated from debugging and is defined as a phase that follows coding. Although it is planned activity, test planning at level 2 may occur after coding for reasons related to the immaturity of the test process. • For example, at level 2 there is a perception that all testing is execution-based and dependent on the code, and therefore it should be planned only when the code is complete. • The primary goal of testing at this level of maturity is to show that the software meets its specifications • Basic testing techniques and methods are in place • Many quality problems at this TMM level occur because test planning takes place late in the software life cycle • In addition, defects propagate into the code from the requirements and design phases, as not review program addresses this important issue. • Post-code, execution-based testing is still considered the primary testing activity	At this level, testing is becoming a planned activity. This implies a commitment to the completion of testing activities. A project planning tool will aid the project manager in defining test activities, allocating time, money, resources, and personnel to testing process. This level of automated software testing is referred to as "<i>incidental automation</i>". At the second level, automated test scripts are modified but no documented standards or repeatability exists. The types of tools used at this level can include project planning tools, capture/playback tools, simulators and emulators, syntax and semantic analyzers, and debugging tools. The introduction automated of automated test tools to a new project is not planned and a process is not being followed. Test design or development standards do not exist. Test schedules or test requirements are not taken into consideration or are not reliable when contemplating the use of an automated test tool. As with Level 1, this type of test automation does not provide much return on investment can actually increase the testing effort.

TMM Level 3

Automated Software Level 3

INTEGRATION

- Testing is no longer a phase that follows coding; rather it is integrated into the entire software life cycle.
- Organizations can build on the test planning skills they have acquired at level 2.
- Unlike level 2 (planning for testing at TMM), level 3 begins at the requirements phase and continues throughout the life cycle supported by a version of the V model (Figure 4)
- Test objectives are established with respect to the requirements based on user and client needs and are used for test-case design and success criteria
- A test organization exists, and testing is recognized as a professional activity.
- A Technical training organization

This level of test maturity is referred to as "intentional automation". At the third level, automated testing becomes both well defined and well managed. The test requirements and the test scripts themselves proceed logically from the software requirements specifications and design documents.

Automated test scripts are created based on test design and development standards, yet the test team does not review automated test procedures. Automated tests become more reusable and maintainable. At this level of automated testing, the return on investment is starting to pay off and a break-even point can already be achieved. The types of tools used at this level include requirements management tools, project planning tool, capture/play-back tools, simulators and emulators, syntax and semantic analyzers, and debugging tools.

TMM Level 4

Automated Software Level 4

MANAGEMENT & MEASUREMENT

- Testing is a measured and quantified process.
- Reviews at all phases of the development process are now recognized as testing and quality control activities.
- Software products are tested for quality attributes such as reliability, usability, and maintainability.
- Test cases from all projects are collected and recorded in a test-case database to test case reuse and regression testing.
- Defects are logged and given a severity level.
- Deficiencies in the test process now often follow from the lack of a defect-prevention philosophy and the porosity of automated support for the collection, analysis, and dissemination of test-related metrics.

This level of test maturity, referred to as "advanced automation", can be achieved when adopting many aspects of the Automated Test Life-cycle management.

This testing maturity level represents a practiced and perfected version of level 3 with one major addition - "post release defect tracking". Defects are captured and sent directly back through the fix, test creation, and regression test processes.

The software test team is now an integral part of the product development, and test engineers and application developers work together to build a product that will meet test requirements.

Any software bugs are caught early, when they are much less expensive to fix. In addition to the tools mentioned at the previous testing levels, defect and change tracking tools, test procedure generation tools, and code review tools are used at this level.

... contd.

TMM Level 5

Automated Software Level 5

OPTIMIZATION, DEFECT PREVENTION AND QUALITY CONTROL

- Because of the infrastructure provided by the attainment of maturely goals at levels 1 through 4 of the TMM, the testing process is now said to be defined and managed and its cost and effectiveness can be monitored.
- At level 5, mechanism fine-tune and continuously improve testing.
- Defect prevention and quality control are practiced
- Statistical sampling and measurements of confidence level, trustworthiness, and reliability drive the testing process.
- An established process exists for the selection and evaluation of testing tools.
- Automated tools totally support the running and re-running of test cases, providing support for test-case design, maintenance of test-related items, defect collection and analysis, and the collection, analysis, and application of test related metrics.

When incorporating the guidelines of the Test Life cycle described in this chapter, and using the applicable tools in an efficient manner, a TMM Level 5 maturity can be achieved. Tools used at this highest level include the ones mentioned within the previous level plus test data generation tools and metrics collection tools, such as complexity and size measurement tools, coverage and frequency analyzers, and statistical tools for defect analysis and defect prevention.

10.16 Human Issues and Challenges in Testing

Those who have spent considerable amount of time in the software industry managing either development projects or testing projects would vouch for the fact the world is replete with love-hate relationship between the developers and the testers! Testing is often not considered a "respected" job. If one thinks what could be some barriers for effective testing, the following factors come to one's mind:

- Optimism: *Belief that the system works*
- Negative attitude towards testing: *a belief that testing is "destructive", testing is equivalent to "fault finding"!*
- Ego: *if "we/I" have coded, it must be perfect!*
- One does not want to fail: *Feeling that the testers will "get" me and prove me wrong!*
- Conflict between testers and developers: *"they versus we" attitude*
- Testing is least structured: *the belief that testing does not require any planning!!*
- The notion that Testing is "expensive": *the belief that testing necessarily requires fancy tools, which cost a lot!*

- Time pressures due to delivery commitments: *we got to deliver first! There is no time for testing—the customer is waiting!*
- Management attitude: *sometime, management does not have the patience; push the product out seems to be the aim!*

While all these are the feelings of people and some of them could be wrongly set pre-conceived notions, one needs to understand that the primary role of testing is to determine whether the AUT meets specification and whether the AUT meets the needs of its users. The secondary role of testing is to instill confidence in the producer of the software system, as well as to provide insight into development process and to continuously improve the testing process.

There are some political (or let's call them "human issues") aspects too. For example—for every software project, there is an interest of conflict that occurs as testing begins (assume that there is no separate test group in case of a small organization and not so large project); the same persons who have built the software are now asked to test the software. This approach, *prima-facie* looks "harmless" i.e. after all, who "knows" the program better than its original developers? Unfortunately, these developers have a vested interest in demonstrating that the software is "error-free", that it "works" according to customer requirements, that it will be completed on schedule and without exceeding the budget and so on... Each of these "vested interests" militates against thorough testing. Unfortunately, in reality, testing occurs at that point of project when deadline pressures begin to rise, progress must be measurable and defects must surface as soon as possible!

There are few more "psychological" aspects that create challenges in the testing activity. Analysis and Design are perceived as "constructive" tasks while Testing gets looked at as a "fault finding" task! This is how it is typically thought: The software engineer creates the software system, its documentation and the related data structures. The software engineer, as a creator, is proud of what has been built through his/her constructive efforts. Come testing and the picture changes—after all, testing is a "destructive" act. Developers feel that in testing, there is a subtle yet a defying attempt to "break" the thing that they have so fondly built i.e. the software developers/programmers tend to have "emotional attachment" to what they have created. So, what happens is (in case the developer is entrusted with the task of testing what he or she had created) that such test cases are designed so as to prove that "it all works"! Actually, what is needed is a third party, independent and "ruthless" approach to testing. While forming the testing teams, all these aspects should be kept in mind. It is worth reading the references [3] and [12] quoted at the end of this chapter; this topic is handled at length by the authors.

10.17 Software Testing Careers and Testing Competence Through Professional Certifications

Software test engineering can provide for an interesting and challenging work assignment and career. In addition, there is high demand in the marketplace for test engineering skills. More specifically, the evolution of automated test capabilities have given birth to many new career opportunities for software engineers. This trend is further boosted by international quality standards and software maturity guidelines that place a greater emphasis on software test and other product assurance disciplines. The aspiration of a person looking for tester's job is best expressed as:

".....I like to perform a variety of different work, learn a lot of things, and touch a lot different products. I also want to exercise my programming and database skills, but I don't want to be off in my own little world, isolated from others, doing nothing but hammering out code. I have an aptitude for working with software tools"

Many software engineers are choosing career in the test arena for two main reasons: (1) the different kinds of tasks involved and (2) the variety of applications for which they are introduced. How does one know whether he/she will make a good test engineer? Good software developers have been trained and groomed to have a mindset to make something work and to work around the problem if necessary. The test engineer, on the other hand, needs to be able to make things fail! At the same time, however, he/she needs to possess a developer's mentality to develop workaround solutions, if necessary, especially during the construction of test scripts. Test engineers need to be structured, attentive to detail, and organized, and, given the complexities of automated testing, they should possess a creative and planning ahead type of mindset. Because testers need to work closely with developers, they need to have the co-operative attitude, high emotional intelligence, be both assertive and poised when working through trouble reports and issues with developers.

Given the complexities of the test effort associated with a client-server or multiple environments, test engineer should have a broad range of technical skills. In addition to this, in an automated test environment, the test engineer needs to know the script programming language of the primary automated test tool. Table 8 outlines a series of progressive steps possible for an individual performing in a professional test engineering capacity. For international certifications in Software Testing, reader may like to visit www.softwarecertifications.com for the details of Quality Assurance Institute's (QAI) CSTE (Certified Software Test Engineer) Professional certification, which is widely recognized. One can appear for this certification examination in India. The examina-

tion is designed to evaluate the knowledge of both the principles and practices of software testing. The URL mentioned above can be visited for information about the knowledge domains covered for the CSTE Certification examination. The table below provides a summary of various roles available for testing professionals.

Table 9: Test Career Progression

<i>Career Progression</i>	<i>Description</i>
Junior Test Engineer	An entry-level position for an individual with a computer science degree or an individual with some manual testing experience. Develops test scripts and begins to become familiar with the test life cycle and testing techniques.
Test Engineer/Programmer Analyst	A test engineer or programmer who has three to four years of experience. Helps to develop or maintain testing or programming standards and processes, leads peer reviews, and acts as a mentor to other, more junior test engineers or programmers. Continues to develop skills in programming languages, operating systems, networks, and databases.
Team Lead	A test engineer or programmer who has four to six years of experience. Responsible for the supervision of one to three test engineers or programmers. Has some scheduling and effort size/cost estimation responsibilities. Technical skills become more focused.
Test/Programming Lead	A test engineer or programmer who has six to ten years of experience. Responsible for the supervision of four to eight personnel. Responsible for scheduling, size/cost estimation, and delivery of product within schedule and budget targets. Responsible for developing the technical approach to the project. Provides some customer support and presentations. Develops technical expertise in a few specific areas.
Test/QA/Development (Project) Manager	Ten-plus years of experience. Responsible for eight or more personnel performing on one or more projects. Full development life cycle responsibility within this area (test/QA/Development). Provides some customer interaction and many presentations. Has cost, schedule, planning, and staffing responsibility.
Program Manager	Fifteen-plus years of development and support (test/QA) activity experience. Responsible for personnel performing on several projects and full development life cycle responsibility. Assumes project direction and profit/loss responsibility.

Conclusion

This chapter discussed some important aspects of software testing. Typically, 25-50% of the total costs of developing a software system are spent

on test activities. This figure is likely to be significantly larger for high reliability, business critical and safety critical systems. Testing is an organizational responsibility. Therefore an appropriate Testing Policy should be defined and documented and Testing Strategy should be evolved. Testing process has its own life cycle. The testing process begins along with the requirements phase of development life cycle and from there runs parallel with the entire development life cycle phases. There are number of testing techniques and testing strategies for the testers to deploy. Extreme testing is a relatively new technique coming up and gaining fast popularity. Towards the end, we've also considered the test process improvement model.

REFERENCES

- [1] *Effective Methods for Software Testing* by William Perry (Wiley)
- [2] *Testing Applications on the Web* by Hung Q. Nguyen (Wiley)
- [3] *Surviving the Challenges of Software Testing* by William Perry and Randall Rice (Dorset House)
- [4] *Software Testing Techniques* by Joe Abbot (NCC Publications)
- [5] *Test Process Improvement* by Tim Koomen and Martin Pol (Addison-Wesley)
- [6] *Software Testing Techniques* by Boris Beizer (Dreamtech Publication)
- [7] *Black Box Testing Techniques* by Boris Beizer (Wiley)
- [8] *Software Testing in the Real World* by Edward Kit (ACM Press—Pearson Education)
- [9] *Software Testing* by Ron Patton (SAMS Techmedia)
- [10] *The Art of Software Testing* by Glenford Myers (Wiley)
- [11] *Automated Software Testing* by Dustin, Rashka and Paul (Addison Wesley)
- [12] *Managing the Testing Process* by Rex Black (Microsoft Press)
- [13] *Testing Object Oriented Systems* by Robert Binder (Addison Wesley)
- [14] *Software Testing & Quality Engineering* (March/April 1999) issue at www.stqemagazine.com
- [15] *Software Engineering: Design, Reliability and Management* by Martin L. Shooman (McGraw Hill International Ed.)

CAREERS IN QUALITY

11.1 Overview

In the chapter on *Software Testing*, towards the end, we discussed career progression for a Software Tester. In this chapter we want to discuss career paths available to quality professional. While doing that, a discussion on some people issues is important as it helps us understand how we "grow". As we have seen so far, Quality (Assurance) is a vast field. It comprises of multiple challenges demanding diverse skills. The professional opportunities are either in "process consulting" (software process improvement initiatives, Auditing and Assessments) or in the area of Testing. Quality Professionals can be deployed in various roles and responsibilities, as we'll see in this chapter. We'll start the discussion with some general issues and then move on to discuss the various roles that quality professionals can play. Each of the roles would demand a certain skill sets. We also provide some useful site references on "quality certifications" at the end of this chapter. Role of a mentor is crucial in helping you develop your career and so we've taken up some discussion on this topic.

11.2 Introduction

An interesting thing to note is how people grow and mature in their professional life. Have a look at where you stand currently as software professional or a quality professional (if you are one). You'll realize that as people mature, they gain a better understanding of themselves and learn to appreciate their own strengths and weaknesses. Having matured, they can set more realistic goals than they were younger, and can adjust career objectives to better fit their potentials and talents.

Initially, we tend to be overtly concerned about status and job titles and we tend to make fuss about the size of our office, cabin, the style of its furniture etc. We tend to frown on this "external motivators" even when we know in the minds of our mind that these are really not the

things that should be make any difference; but we do sulk about all these things.

Throughout their careers, professionals learn from their successes and failures. Successes build self-confidence but defeats also teach you a lot and it is ok to have a few of them. They provide us evidence of our shortcomings and limitations. In the famous book "Personal Software Process", Watts Humphrey says: *"..... as they debug their programs, software engineers constantly face the consequences of their own fallibility. When engineers learn to realistically accept themselves, they are better able to establish rewarding career goals....."* This is very well holds for the quality professionals as well.

11.3 P-CMM and Careers

One point to note is that the People Capability Maturity Model (famously known as the P-CMM model) is not going to be discussed here. It is a generic framework for improving people related practices in organizations. It has a number of Key Process Areas that have to do with career development, empowerment etc. The interested readers are directed to SEI's site www.sei.cmu.edu to get overview and other details of this model. However, since we are discussing careers of software quality professionals in this chapter, it would be pertinent to talk about some people issues. Some of the top people issues as perceived by the industry are the following:

- Enabling people to deal with continual change in the organization
- Managing skills and career growth
- Ensuring consistent communication between managers and staff
- Providing clear performance feedback
- Overcoming low morale & burnout
- Subjective or inappropriate measuring
- Identifying competencies
- Defining roles and responsibilities
- Motivating personal goals to align with organizational goals
- Reducing turnover

11.4 Some Important "People Issues"

Each of the issues mentioned above is important for an organization to manage if high turnover of people is to be helped. For example, let us take the issue of "low morale & burnout": it is the phenomenon of job withdrawal. An employee's feeling of being helplessly trapped in a meaningless job generally causes burnout. It is not caused by age, overwork, or exhaustion but is more a defense against the loss of self-esteem caused by an apparent lack of personal value. Routine jobs with little challenge evoke reduced performance; one dull task inexorably follows

Other people issues are: Age and Creativity, Age and Performance, Age and Motivation. Interested readers are referred to [2]

3.5 Finding a Mentor to Shape Your Career

We'd like to start this section by making a note that "Mentoring" is one of Key Process Areas in the P-CMM model from Software Engineering Institute (the SEI—for additional details, visit the site quoted in section 3.3 www.sei.cmu.edu). The word "mentor" comes from the mythical character *Mentor*, wise man and trusted friend of Odysseus in Homer's *Odyssey*. A mentor provides advice based on experience, helps you navigate around personal and professional obstacles, and acts as a mirror to enable you to see yourself more clearly. A true mentor is honest and would tell you what he/she frankly thinks whether or not the protégée (the person under mentoring) wants to hear it. At times, a mentor would raise protégée's self-confidence by asking his advice. The most effective mentors encourage you to test your wings and fly. They cheer your success and introduce you to others who might be able to help you soar to higher levels. Most importantly, they tell you what you *need to hear* and not what you *want to hear*.

It is worth noting that it takes a secure person to be a good mentor, someone who can honestly say, "I taught her/him everything I know" instead of "I taught her everything she knows." It is important to understand that being a mentor is a very tough mantle and not all managers are capable of providing mentorship. A common mistake is to assume that the mentor has to be a technical expert in your area of work; this is not true. Good mentors are special people—as said earlier they help to protégé to fly. So look everywhere, not just in the obvious places like your workplace. You may find potential mentors at work, in your family, or in your community. Look for people with complementary skills to yours. For example, if you are new test lead, maybe you'd benefit from a mentor with extensive experience as test manager. If you're a fresh tester, perhaps someone with contrasting test skills could help you expand your range of testing skills. Someone can be a good mentor on a single subject; mentors don't need to know more than you do about all aspects of your job.

Beware of choosing mentors only for the power they wield. It is far better to spend time with an individual contributor who possesses a great deal of wisdom than a VP whose only attribute is the influence she could use to get you a promotion. What you learn from a wise person will last a lifetime! What you get from a powerful person will only last as long as their influence on your career lasts. Search for people with whom you have something in common, whether it is shared values, gender, ethnic background, or sense of humour. Having a connection to your mentor beyond your desire to learn from him/her will help establish and sustain the mentoring relationship. To

summarize, a mentoring relationship is about spending time getting to know someone who has the wisdom and experience that can help you in the long term. It is not about just finding a job; it is about shaping a career.

11.6 Roles for Quality Professionals

At the outset, we would like the readers to once again note difference between Quality Assurance and Testing. Quality Assurance often gets mixed with testing, which is essentially "quality control" (QC). William Perry (author of the famous book—"Effective Methods for Software Testing") provides a correct viewpoint; "...the role of QC (Quality Control) is to measure product against a standard or attribute...." Thus, according to William Perry, the purpose of QC is to identify defects (for the developers to correct) so that a defect-free product or a product within acceptable quality can be produced. Quality Assurance (QA) is separate from QC and deals with the overall management of quality. Often, we hear the statement that the product is in QA—this can be misleading. In this context, QA is projected as quality control or testing. This error is not limited to day-to-day users or developers but even to people dealing with processes. For a more detailed discussion on this, readers can refer to [3].

With this important point in the background, presented below is the list of roles/jobs typically available for quality professionals; followed by brief discussion on each of them. Since these roles require you to work with the developers, management teams within your organization as well as the customer, you'll find that good communication skills become an essential skill requirement in all these roles discussed below. It is to be noted that the following are only representative and not exhaustive titles for the typical roles available for Quality professionals in the Information Technology/Software industry. Also, please note that the following roles are not necessarily hierarchical.

- System Tester
- Software Quality Tester
- Quality Assurance Engineer
- Process/Quality Engineer
- Quality Control Analyst
- Quality Assurance Auditor
- Quality Control Manager
- Software Quality Assurance Manager
- Senior Product Verification Specialist
- SEI CMM Lead Assessor
- Director, Software Quality Assurance

11.6.1 System Tester

Sometimes such a role is offered on a contract basis and could be of short duration. Such a role may come into existence when multiple software applications are in various stages of development and require functional as well as data and database integrity testing prior to implementation. Functional and database integrity testing is generally automated through appropriate test software. This kind of role calls upon a number of corporate and external relationships:

- Daily contacts with development and services groups
- Report Regular contact with customers and audit agencies
- Daily involvement with staff
- Reporting to the Operations and Services group

Your responsibilities in this role is mainly to work under the direction of the Project Lead and with additional assistance from the Software Quality Assurance Lead and project development teams to undertake the following tasks:

- Writing or assisting in writing the Test Plan
- Creating test cases for all identified components
- Automating test cases using suitable tool(s)
- Executing test scripts using suitable tool(s)
- Providing written test results

As a System Tester, you're expected to provide the following deliverables

- Provide Test Cases stored as a Test Suite
- Provide Test scripts that are modular and maintainable
- Provide written test results that are reproducible

To perform this role effectively, you should have at least 2-4 years of experience in planning and organizing testing efforts for large systems in GUI and non GUI environments including the execution of systems integration tests and specialized tests (i.e.) stress tests etc., experience in planning, organizing and implementing testing efforts at the specified levels of proficiency, possess knowledge of structured methodologies for the development, design, implementation and maintenance of applications, knowledge and experience in the use of case tools and data query tools to aid in the development of test cases. Experience in the use of automated test tools and version control systems on one or more platforms, excellent analytical, problem solving and decision-making skills; organizational, creativity and perseverance skills; verbal and written communication skills; and interpersonal skills

are also required. You should be a team player with a track record for meeting strict deadlines.

11.6.2 Software Quality Tester

Most software companies have a QC group whose job is to test the software for bugs and incompatibilities with specifications. QC professionals are responsible for designing testing strategies that will ensure a quality product, and for implementing these strategies.

For performing this kind of role effectively, some programming experience is necessary, but not as much as with traditional programming. Many companies hire entry-level people into QC, as a way for them to get to know the product and the working environment. Communication skills, teamwork abilities, and problems solving skills are important. Such candidates are thoroughly groomed into organization's testing procedures.

A position like this, is often a stepping stone to other positions in the company, such as technical support or programming. Apart from this, an aspiring candidate should note that QA is related to the Total Quality Management (TQM) movement, and there is a lot of literature written about various QA strategies. People who become heavily involved in QA and knowledgeable about it are highly valued in a company.

11.6.3 Quality Assurance Engineer

This job would require demonstrated experience in writing and performing detailed reviews of procedures and instructions. The person is expected to be experienced in the review of technical procedures required to perform and control activities associated with design, procurements, of QC Tools, inspection and testing of items and systems associated with Codes and standards.

11.6.4 Senior Quality Assurance Engineer

Responsibilities include participating in high-level design reviews; developing test specifications, test plans, test cases and automation scripts for new product functionality; functional, integration and regression testing of the modules and identifying and documenting product effects. Additional experience in developing a QA infrastructure, methodology, and process are crucial to this role. This position would typically require at least 4+ years of industry experience, working in positions of increasing responsibility. Test automation experience is mandatory, in particular the execution of tests using some test automation tools. These days, experience in testing Web applications and knowledge of Web technologies, such as HTTP/HTML and XML is an asset, and so is understanding of RDBMS and SQL. The persons should

have excellent written and verbal communication skills and the ability to work independently as well as in a team environment.

11.6.5 Process/Quality Engineer

In this role you will identify, lead and support process improvement activities leading to performance improvement and customer satisfaction. Hands on application of process map and effectiveness of the quality system in the development process improvement methods, structured problem solving and statistical analysis techniques will direct your focus. You will also play a key role in transition from ISO9001 to other quality models as required by your client's evolving business and other requirements. A strong belief in teamwork, commitment to continuous improvement, previous experience with quality systems and related technical education will help you meet the interesting and challenging nature of this position.

11.6.6 Quality Control Analyst

As a Quality Control Analyst, you will ensure product quality by testing its security features as well as its interoperability with the various security providers. In this job, you'd need strong technical skills, you should also have a working knowledge of security technologies and enjoy working closely with development to produce a quality product. You'd be working with Development and Product Management to understand testing requirements. You'd also develop and execute manual and automated test packages. As your additional responsibilities, you would be required to report and follow up on product issues and also assist with development testing activities. The skill requirements for this role would be college diploma OR university degree in Computer Science or equivalent, minimum of 2 years testing experience, demonstrated experience testing product security and excellent communications skills (oral and written).

11.6.7 Quality Assurance Compliance Associate

In this role, responsibilities would include: ensuring that development processes, specifications, product packaging and product release program remain consistent with the Product Registration procedures in the region of work where employed. Quality professional in this role is also expected to advise the product development organization on expectations regarding validation, assisting in the resolution of product-specific process issues. Additional responsibilities would include participating in corporate audits and providing status reports as required, maintaining quality metrics for the product development organization, encouraging continuous improvement, assisting in

the management of the change control program, assuring Product Complaints are investigated in a timely manner; participate in new product introductions.

Employer would look for a person with a university degree in a relevant scientific discipline, superior English communication skills, working knowledge of standard development practices with a minimum 5 years of experience in the industry, ability to work effectively with regulatory agencies, technical and non-technical personnel to provide products of superior quality.

11.6.8 Quality Assurance Auditor

Experienced Internal (or external) QA Auditors in utility, process or manufacturing environments familiar with one or more of the standard QA programs like ISO 9000. Interpersonal skills and judgment for dealing with senior management would be considered important aspect of the job.

11.6.9 Quality Control Manager

In this role, your responsibilities include reporting to the Director of Quality, assisting the organization in achieving corporate objectives through the effectiveness and cost-efficiency of product quality controls to ensure on-going compliance to product performance parameters, including regulatory and contractual requirements, ensuring that product Quality Control requirements are defined and controlled by driving the evolution of applicable systems, in a multi-site ISO 9001 environment, and coaching cross-functional personnel on product assurance.

11.6.10 Senior Product Verification Specialist

As a Senior Product Verification specialist, you will be expected to have over 5 years of experience in Software Verification. You should possess excellent knowledge of the design life cycle and extensive expertise in software verification practices. As an experienced tester, your duties will include designing test plans and executing test cases as well as contributing to the overall test strategy for the product. You will be expected to act as a resource person for the group reporting to you. This includes training and mentoring newcomers, providing support to team members, assisting your manager in planning overall resources and tracking projects. As a key member of the Product Verification Team, you will require self-direction, good multi-tasking abilities, excellent troubleshooting and reporting skills.

This role calls upon some personal qualities like the ability to work independently; flexibility and ability to adapt to a new environment; excellent communication skills (verbal and written). Team orientation;

taking initiative; being a quick learner; solid knowledge of and experience with: Product Verification Methodologies (Strategy, Test plan, Test cases writing/execution, problem reporting), overall Software Development process and life cycle; Black box, load, installation and performance testing are also the essential skills.

11.6.11 Software Quality Assurance Manager

For this position, the employer organization typically looks for enthusiastic Software Quality Assurance Manager to lead their QA/Test group in the delivery of software of the highest caliber. As QA Manager your mandate is to evaluate the performance, interoperability and reliability of developing organization's software and ensure that product specifications and customer requirements are met. As a QA Manager, you will lead the planning and development of your organization's ongoing test automation effort; manage the development and implementation of test plans and test cases; form SQA plans, processes and procedures and advise the development team of quality issues. This job would typically insist on the following minimum skill requirements on part of the candidate:

- Minimum of 4 years QA Management experience in the software industry
- Degree in Computer Science/Mathematics/Engineering or related discipline.
- Must possess experience with software quality assurance tools, methods, automation and performance testing.
- Knowledge of eminent tools (for example Winrunner, SILK)
- Exceptional written and oral communication

11.6.12 SEI CMM Lead Assessor

This is a fairly senior role. You're expected to be a talented and team oriented professional to lead a group towards process improvement/maturity practices, act as a mentor and a leader who drives the initiative across the organization in close coordination with the SEPG/Software Process Improvement group. The individual must have successful previous implementation and assessment experience. In this role, your typical responsibilities include: Preparing the detailed Software Process Improvement (SPI) plan with associated risk and opportunities, Conducting organizational assessment of current maturity level, Facilitating organizational change management through SPI, Defining and driving SEI CMM assessment plan for the organization that enables SEI CMM level 2-5 compliance to the required level of the SEI CMM Model (details of this and other process improvement frameworks are discussed in a separate chapter devoted to this topic), Defining metrics and processes necessary

for measurements of SEI CMM compliance as an organization, Performing SEI CMM assessments for IT organizations required level of compliance, Performing SEI CMM audit functions to insure continued compliance.

To fit well in this demanding senior role, you need to be (preferably) a SEI Certified CMM Lead Assessor having the experience of assessing both large and small organizations for SEI CMM compliance. The role expects you to have solid knowledge of Capability Maturity Model. It is essential to have project leadership experience in IT organization and also experience with software development methodologies. Since quantitatively managing the software process is the crux of the requirement of the SEI CMM model, you must have experience in establishing metrics for SEI CMM organization measures. In this role, SEI CMM audit experience for IT organizations and needless to say like all QA responsibilities, you should also have excellent communication and interpersonal skills.

11.6.13 Director, Software Quality Assurance

A QA Director is expected to lead the Quality Assurance group/department. Past experience with SQA stints helps to guide the Software Quality Assurance professionals that form the QA group in the organization. In some organizations dealing with e-Business applications, a person in this position is expected to be a hands-on manager of software quality assurance for large-scale high volume web sites. The successful candidate must have a proven track record in building a software quality assurance group. Practical experience with industry leading tools, writing and executing test plans is a must. This position would typically report to the VP—Software Development. Minimum skill requirements for this job would be: at least 2 years experience building and managing a software quality assurance department, 5 years in a practical hands-on software quality assurance role, experience with a few industry standard tools, experience in stress testing Internet sites, some good degree in software technology.

11.7 Quality Certifications

In this section, we'll discuss the various international level certifications available to persons who wish to work as quality professionals. It should be noted that these certifications are only meant to endorse your proficiency in the concerned area. However, in themselves, they are not a substitute to work experience in quality profession. It is the experience that can really build the knowledge skills and competency. While Certifications help you to gain the conceptual knowledge, it is the field experience that makes you a real practitioner/SME (subject matter expert). This happens when you get the opportunities to apply the

knowledge gained through certification. Needless to say, however, that these certifications go a long way in bringing in the required conceptual framework which you can hope to apply in your working situations if you have not already got the opportunity to do so. Pl keep this in mind while you read ahead.

We are going to discuss the certifications available from two well known international level organizations; ASQ (American Society for Quality) and QAI (Quality Assurance Institute—USA). At the end, we've provided the URLs from where the interested readers can seek further details.

11.7.1 Certification Benefits

Certification is a mark of excellence that:

- Provides verification of an individual's knowledge
- Is recognized by several organizations
- Is maintained through demonstrable professional practice
- Ensures continuing education
- Is reinforced through use of designator letters.

Certification has a certain impact on the individual who undergoes the certifications and on the organizations that sponsor individuals for the said certification and the professional community around. Impact on the person is high immediate and on-going professional growth. Impact on the organization is that it highlights interest in Quality, infects colleagues and creates ground swell of professional activity. Impact on our community is that of heightened professionalism that spills over to other organizations. Thus, there is no doubt that Professional certification has long been a way to advance one's career independent of industry and political constraints.

11.7.2 Certifications from American Society for Quality (ASQ)

An ASQ Certification is a peer recognition program indicating proficiency in a designated subject area. These programs are independent of employers and hence portable. Certification is indicated by using the appropriate letters after the person's name e.g. Ninad Deshpande, CQE. Table 1 provides a summary of quality related Certifications available from ASQ. For those who'd be interested, we've provided below a brief history of certifications from the American Society for Quality.

History of ASQ Certification Programs

Table 1: ASQ Certifications

ASQ Designation	Designed For ...
Certified Quality Engineer (CQE)	... those who understand the principles of product and service quality evaluation and control.
Certified Quality Auditor (CQA)	... those who understand the standards and principles of auditing and the auditing techniques of examining, questioning, evaluating, and reporting to determine quality systems adequacy.
Certified Quality Manager (CQMgr)	... those who understand quality principles and standards in relation to organization and human resource management.
Certified Software Quality Engineer (CSQE)	... those who have a comprehensive understanding of software quality development and implementation, have a thorough understanding of software inspection, testing, verification, and validation; and can implement software development and maintenance processes and methods.
Certified Reliability Engineer (CRE)	... those who understand the principles of performance evaluation and prediction to improve product/systems safety, reliability, and maintainability.
Certified Quality Technician (CQT)	... those who can analyze quality problems, prepare inspection plans and instruction, select sampling plan applications, and apply fundamental statistical methods for process control.
Certified Mechanical Inspector (CMI)	... those who, under professional direction, can evaluate hardware documentation, perform laboratory procedures, inspect products, measure process performance, record data, and prepare formal reports.
Certified Quality Improvement Associate (CQIA)	... those who have a basic knowledge of quality tools and their uses and are involved in quality improvement projects but do not necessarily come from the traditional quality area.
Six Sigma Black Belt Certification	... those who have a comprehensive understanding of Six Sigma and its methodologies.
Quality Auditor Certification-Biomedical (CQA-Biomedical)	The ASQ Board of Directors and the Certification Board with sponsorship from the Biomedical Division approved this most recent "Add-On Certification" to the Certified Quality Auditor exam. Note: You must be an ASQ Certified Quality Auditor and have two years of experience working with Biomedical auditing. This two-year requirement can be part of the eight-year requirement used for the CQA exam.
Quality Auditor Certification-HACCP (CQA-HACCP)	In November 1999, the ASQ Board of Directors and the ASQ Certification Board, with sponsorship of the Food, Drug, and Cosmetic Division, approved the first "Add-On Certification" to the Certified Quality Auditor exam. Note: You must be an ASQ Certified Quality Auditor and have two years of experience working with the HACCP standards. This two-year requirement can be part of the eight-year requirement used for the CQA exam.

The certification program was first developed to serve three industries; Nuclear, Consumer and Automotive. Certification programs are established to meet specific quality focus areas and are typically developed over a 2–3 year period in consultation with recognized industry experts, for example:

- Certified Quality Engineer (CQE) was started in 1966.
- Certified Quality Auditor (CQA) was started in 1987.
- Certified Quality Manager (CQMgr) was started in 1995.
- Certified Software Quality Engineer (CSQE) was started in 1996.
- Certified Quality Improvement Associate (CQIA) was started in 2001.

11.7.3 Certifications from Quality Assurance Institute

11.7.3.1 Certified Software Quality Analyst

Acquiring the designation of Certified Software Quality Analyst (CSQA) indicates a professional level of competence in the principles and practices of quality assurance in the IT profession. CSQAs become members of a recognized professional group and receive recognition of their competence by business and professional associates, potentially more rapid career advancement, and greater acceptance in the role as advisor to management.

11.7.3.2 Certified Software Test Engineer

The Certified Software Test Engineer (CSTE) certification is intended to establish standards for initial qualification and provide direction for the testing function through an aggressive educational program. Acquiring the designation of Certified Software Test Engineer (CSTE) indicates a professional level of competence in the principles and practices of quality control in the IT profession. CSTE become members of a recognized professional group and receive recognition of their competence by business and professional associates, potentially more rapid career advancement, and greater acceptance in the role as advisor to management.

Conclusion

In this chapter, our aim was discuss about growth path for quality professionals. Broadly speaking, as a quality professional, you can grow either in the process side (Quality Assurance) or testing side (Quality Control). To shape your career you need good planning as well as a strong mentor. Armed with appropriate certifications your path to upward mobility can be fuelled for a faster growth. Though there is no substitute for hard work and field experience, certifications can help

you expand your vision; they also help you with the right conceptual framework.

REFERENCES

- [1] *Surviving the Challenges of Software Testing: A People-oriented Approach* by William Perry (Dorset House Publishing)
- [2] *Managing Technical People* by Watts Humphrey (Pearson Education)
- [3] *Process Quality Assurance for UML-Based Projects* by Bhuvan Unhelkar (Addison-Wesley)
- [4] www.softwarecertifications.com for information on certifications CSQA and CSTE
- [5] asq0407@quality.org for the details of ASQA Certification Programs

PROCESS IMPROVEMENT RELATED MISCELLANEOUS TOPICS

A.1 Some Basic Terms

In this section, we have discussed some fundamental terms that were mentioned in Chapter 9 (*Process Improvement Models*)

A.1.1 Process

The term "process" can be defined in a variety of ways; here are a few acceptable definitions:

'a course of action or proceeding, especially, a series of stages in manufacturing or some other operation'

'a system of operations in producing something...a series of actions, changes or functions that achieve an end result'

'a sequence of steps performed for a given purpose, for example the software development process'

'a collection of activities that takes one more kinds of input and creates an output that is of value to the customer'

'a set of actions, tasks, and procedures that when performed or executed obtain a specific goal or objectives. More specifically, a software process is a software development procedures'

'the set of activities, methods, and practices used in the production and evolution of software' [SEI CMM]

The three important aspects of Process are shown in Figure 1. Next, we discuss about symptoms and behaviours of "process orientation". This discussion is important for understanding the patterns the "High Maturity Organizations". (In Chapter 9, there is a discussion on "practices followed at "Mature Organizations".)

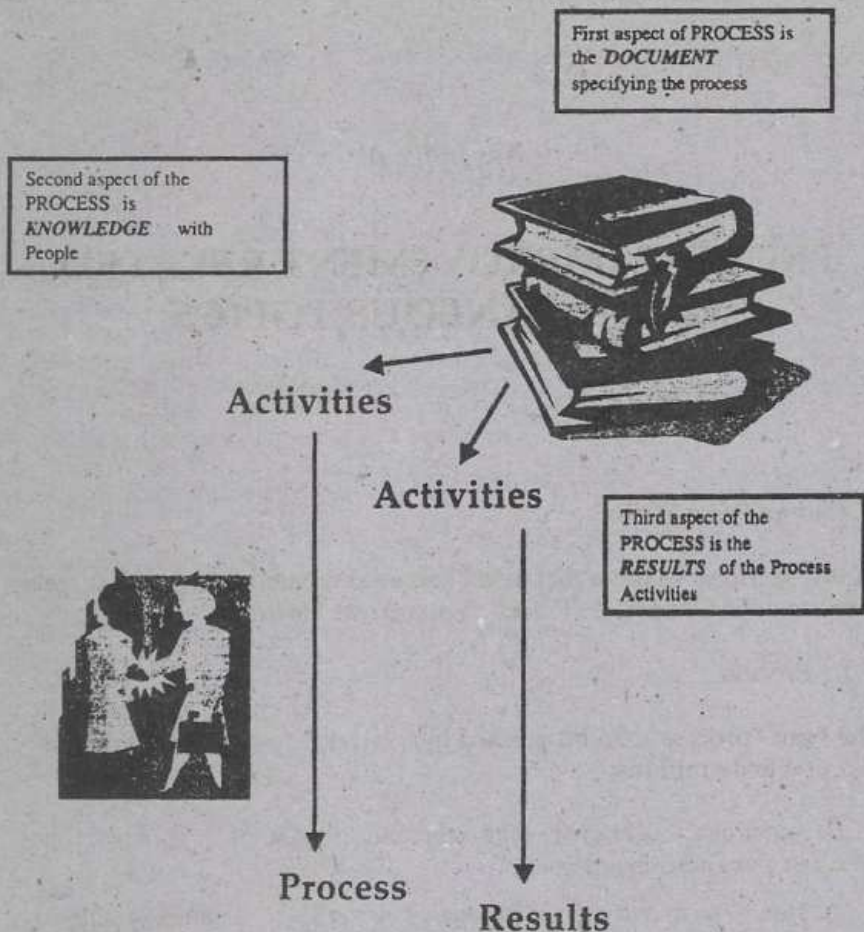


Figure 1: Process Aspects

A.1.3 Maturity

Readers must understand that throughout the discussion of the term "process improvement" and related topics, the term "maturity" is used in the context of "process maturity". Organizations at various levels of process discipline are like people at different stages of maturity (obtained out of the process of living which provides experiences and lessons!). The difference between an immature process and a mature process is like the difference between a young child and a mature adult. Process thinking has an impact on behaviour. For illustrations of this point see Table 1 (adapted from [3]). Also another good reference is [5] for concepts on "discipline"—applicable to Software Engineering Profession—as provided by Watts Humphrey.

Table 1: Process Focus

Parameter	Without Process Focus	With Process Focus
Process Discipline	Processes are improvised by the staff and their management on the fly (in the course of performing the activities)	Processes are defined and followed by staff as well as by management; process discipline is the un-questioned norm
Organization	Functions and roles do not necessarily align with the process requirements	Functions and roles are defined in support of the process
Management	Staff performance is measured in terms of the number of work hours (irrespective of whether they were productive or not) Managers are usually focused on solving immediate crises (fire-fighting approach)	Staff performance is measured in terms of process performance and results rather than merely focusing on activities. Such measurements are defined and agreed Managers focus on the quality of processes as well as quality of products
Skills and Training	Training is <i>ad hoc</i> , and oriented towards personal inclinations	Training is a planned activity, subject to periodic reviews by all concerned
Tools and Technology	Tools & Technology are acquired as <i>ad hoc</i> point solutions without an overall plan and strategy	Tools & Technology acquisition involves all stakeholders (in cognizance to organizations vision and mission) Selection is made in support of the end-to-end process and the automation of process activities with a view to obtain higher productivity in the long run

A.1.4 Process Orientation

An effective process environment is characterized by an existence of process feedback mechanisms in order to collect data on process performance. This feedback data collected and stored in process data becomes organization's memory ("Lessons learnt") wherein the organization stores experiences and results of its process performance. This can then be used to improve processes (for example improving the performance of another similar projects executed next time.) Such information should be available to all those in the organization who are involved in performing process activities (say software engineering). Table 2 shows symptoms and behaviour in the context of process orientation.

Table 2: Process Orientation

Key Term	Description of symptom/behaviour
Process Visibility	The process definition and process responsibility are visible to everybody in the organization
Process Discipline	Following the process is the norm, and acting outside the process is the exception
Process Institutionalization	Following the process is "second nature" to the people in the organization i.e. following the process is engraved into the organizational policies and procedures, and supported by the management
Management Commitment to Processes	The processes are supported by top management of the organization as well as all other levels of management and staff
Process Enforcement	Process enforcement is evident and is in force
Process Ownership	The process is owned, maintained and continuously improved and supported by an organizational infrastructure
Process Feedback	Feedback on the process effectiveness is practiced by every one in the organization, and is enabled by appropriate feed back mechanism
Performance Assessment	Measurement and evaluation of staff and team performance is tightly linked to the process performance (i.e. achieving the process goals)
Process Training	Process Awareness and training mandatory for all staff, and process induction is mandatory for new recruits (for that matter for anyone who is engaged in software engineering activities)
Process Improvement	Process Improvement in planned and implemented through the participation of all the staff involved in software engineering activities as well as relates support processes (configuration management, training etc)

A.2 High Maturity Organizations (those at SW CMM Level 4 or 5)

In Chapter 9, we discussed the term "maturity" in terms of "software processes". It will now be interesting to understand the practices followed in mature organizations. Over the last few years the Software Engineering Institute has participated in several workshops and site visits with maturity level 4 and 5 software organizations. This section summarizes the lessons learned as reported by SEI based on their interactions with high maturity organizations. Specific areas of interest include *statistical process* and *quality control* and product lines/families, and several other observations made by the SEI over a variety of engineering and management practices, including issues outside the scope of the Capability Maturity Model for Software. For a detailed discussion, reader is referred to [1]. The purpose of this section is to

summarize in a general way the techniques, methods, and lessons learned about getting to CMM levels 4 and 5. We do not, however, attempt to describe all good software engineering and management practices. Although these observations are specific to software organizations, it is believed that these practices, or discipline-specific variants, are valuable to any high maturity organization in any engineering discipline. The data that is summarized in this section comes from a variety of sources, including published case studies, workshops, site visits, and assessments, data collected by SEI through their surveys etc.

It turns out that *high maturity organizations* typically have a broader scope of improvement concerns than just the CMM's process issues. Some high maturity organizations were found to be doing process improvement long before the Software CMM framework was published. Others, such as Motorola India, were started with one business objective being "high process maturity." Common Practices Observed in High Maturity Organizations are described below.

A.2.1 Role of TQM in Mature Organizations

If there is a Total Quality Management (TQM) initiative within the company, the high maturity organization's software process improvement program is explicitly aligned and co-ordinated with the TQM initiative. Aligning with TQM initiatives can be a challenge for high maturity organizations. If the organization is already performing at Six Sigma levels, there is a major challenge in improving dramatically. In such cases, the alignment may be conceptual rather than quantitative.

A.2.2 Role of ISO 9001 Certification

Eleven out of thirteen of the organizations surveyed (The 1999 survey conducted by the SEI) had ISO 9001 certification. Some commented that they began their process improvement efforts using ISO 9001, then shifted to the CMM after obtaining certification. Others obtained certification as a business requirement in their market.

A.2.3 Addressing Other Issues

As mentioned earlier, surveys show that high maturity organizations address issues outside the scope of the CMM. Their improvement programs include a strong emphasis on automating the software process and addressing people issues. Processes, data collection, and statistical analysis are automated wherever practical. High maturity organizations generally emphasize openness, communication, and a commitment to quality and the customer at all levels. They appreciate the "people issues" (In this connection, it is worth visiting the SEI site (www.sei.cmu.edu) to fully understand the overview of the "People

CMM" model apart from the mention in this chapter). Thus, one can say that much of the maturity practices are to do a lot with "organizations culture".

A.2.4 Process Orientation in High Maturity Organization

Another finding of these surveys conducted by the Software Engineering Institute is that the high maturity organizations encourage a "process orientation" in their staff. Worker empowerment and participation in process definition and improvement activities are real; process improvement is part of everyone's job. Their strategic approach to quality management involves the linkage of quality with business goals and a focus on customer satisfaction and delight. (Interestingly, the "People CMM" model addresses several of these aspects.)

A.2.5 Quality Culture

There is a "quality culture" in high maturity organizations. Rewards and incentives are established for process improvement efforts, and worker empowerment and participation are more than just slogans. People believe in the process, and when mistakes happen (as they inevitably do in any human endeavor), the focus is on improving the process, not disciplining or penalizing the people, who may only be the bearers of bad tidings.

A.2.6 Approach to People Issues

High maturity organizations recognize the importance of good staff. To quote one participant in the SEI CMM 1996 workshop, "Getting the right person into the right job on the project is still the most important aspect of project success. People are not plug-compatible. The expertise of individuals is critical. Process is an enabler; not a replacement...". Knowledge and skills are systematically cultivated in software engineering, management, interpersonal skills, and the application domain. Another workshop participant stated, "We have the philosophy of not assigning people to jobs they are not prepared for...." It was also observed that not everyone needs to have deep domain knowledge so long as the key lead engineers do, although at least one member of each team needs to have the right domain expertise to support the team effort.

A.2.7 Approach to Using the CMM Model in High Maturity Organizations

High maturity organizations apply common sense in using the SW CMM. New projects are typically expected to address all 18 key process areas, but alternate implementations are used when appropriate.

When there is a controversy, (strategic) business value is the foremost factor in resolving the situation; this sometimes means going against the "letter of the CMM" or customer desires, but doing so with data to support an informed decision.

A.2.8 Customer Relationships and Customers' Maturity Levels

It is very difficult to be a high maturity supplier if you have a low maturity customer (or low maturity suppliers or partners in the case of strategic alliances, joint ventures, and virtual organizations). At higher maturity, there is a better foundation and understanding to explain to customers why you do things a certain way. The customer is usually reasonable when presented with data, facts and frequent patterns of success, e.g., delivering on schedule and budget. The general philosophy of high maturity organizations is to proactively work with the customer to gain a mutual understanding of what will be done, which often means educating the customer on how the organization usually does things, with data and results to back up the methods.

One high maturity organization provides the last two years of performance data in its proposals; although it is rarely the low bidder, consistent past performance provides credibility. In one instance, a long-term customer in a maintenance environment did not want the software development plan updated. The supplier chose to develop a separate plan that the organization uses—an example of doing the right process thing, even if the customer is uninterested (or perhaps opposed!). Another supplier, in a commercial environment, established a customer liaison engineer, separate from the systems engineer, to work the interface with the customer. The customer liaison engineer is usually on the customer's site, and change control boards include customer representatives.

The SEI 1999 survey indicates a general emphasis on proactively managing the evolution of the customer's requirements via evolutionary and incremental life cycles. Eleven of the thirteen organizations surveyed use incremental life cycles, and nine of the thirteen (with two piloting) use evolutionary life cycles.

A.2.9 Project Management (Issues for Maintenance Organizations)

A practice in the CMM's project management key process areas that requires significant interpretation for maintenance organizations is size estimating. Maintenance organizations are frequently funded as level-of-effort, and little business value may be derived from size estimating. Although size may be a useful measure for enhancements, for corrective maintenance "size" is likely to be "number of problem reports," and the variability is likely to be quite high. All thirteen organizations in the survey (conducted by the SEI in 1999) use lines of code as a size

measure (and two use function points also, with two more organizations piloting them), but comments pointed out that there are instances where size estimating is not of value for maintenance—schedules and budgets may be fixed *a priori* for maintenance projects, and the variable parameter is the functionality of the update package. It is also interesting to note that only eight of the thirteen organizations surveyed use cost models in their planning process, and one maintenance organization had actively rejected them.

A.2.10 Risk Management in High Maturity Organizations

High maturity organizations systematically manage risks. Eleven of the thirteen organizations surveyed do “systematic” risk management. This would lay a good foundation for transitioning to the CMM-Integrated model where “Risk Management” is a separate process area at Level 3. Based on the survey the following characteristics can be noted for the high maturity organizations:

- a) Operating in a high risk environment;
- b) Having a profound knowledge of the risks associated with their (fairly specialized) application domains; and
- c) Having internalized risk management so thoroughly—and systematically—into their processes that they no longer recognize how mature their risk management process is.

The conclusion appears to be that systematic risk management is an intrinsic characteristic of high maturity, which is integrated into the process and internalized by the staff.

A.2.11 Roles, Responsibilities and Delegation in High Maturity Organizations

In high maturity organizations, many traditional management responsibilities are delegated to roles specified in the process, e.g., progress tracking, day-to-day customer interfaces, and process improvement. Managers are freed to focus on longer-term strategic and operational issues. Managers can truly “manage by exception” in an open culture where the messenger is not shot and problems are identified and escalated appropriately and quickly. When roles are established, and responsibility and authority delegated, the people in the roles:

- speak for the organization
- are empowered to make commitments that will be honored
- are supported by their management

A.2.12 Measurement in Mature Organizations

High maturity organizations tend to have dedicated measurement people at the project and organization levels, although they may not be assigned full-time. They also encourage their customers to become actively involved in specifying measures and setting quality goals. Measurement in a high maturity organization can be characterized as:

- driven by business goals
- standardized across the organization for common measures
- tailored to the specific needs of the user/customer
- based on operational definitions that define how to collect consistent data
- collected as close to the point of origin as possible (primary metrics)
- involve the active participation of the affected parties, including the customer

A.2.13 Product and Process Assurance in High Maturity Organizations

Software Quality Assurance (SQA) is perhaps the most controversial key process area in the CMM. There are passionately held, opposing opinions on whether there should be an independent SQA organization or whether the SQA function should be "built into the process" as part of the quality culture that should be expected of high maturity organizations. (Reader will re-collect that the topic of how the organization and reporting of the SQA group should be was discussed in the early chapters of this book; with this background, it would be interesting to see what the survey shows about "high maturity organizations")

Questions in the survey were intended to probe this issue, but the surprising result was that all thirteen organizations indicated that they use an "independent SQA group" and embed the SQA function in the process. The SQA groups in these organizations were found to use the level 4 process and product data to identify high-leverage opportunities for auditing.

As one reviewer commented, "I was convinced several years ago that SQA would disappear as an organization matures and software quality functions would be embedded into the software engineering functions. I viewed the CMM to have a problem, because SQA as an independent function was always required even as the organization matured (since it was a Level 2 KPA). However, I changed my opinion after our experience ... When SQA makes the appropriate transition to process audits in addition to product audits, they can become a valuable team member. We still make mistakes in a high maturity organization, and it is still valuable to have a backup to catch those. Their role in process audits is very valuable as it separates that function from the SEPG so that the SEPG does not appear to be the policeman."

Further to this, the SEI Survey (1999) found that some high maturity organizations separate process and product assurance. The "SQA group" may focus on process monitoring, while product assurance is built into the peer reviews and configuration management system.

A.2.14 Training and Mentoring in High Maturity Organizations

High Maturity Organizations give extraordinary importance to Training. Some organizations have mandatory training for new hires that can last up to eight weeks, plus mandatory continuing education requirements. This training includes internally and externally developed training materials, awareness programs, and workshops. Other organizations rely heavily on formal mentoring programs to impart skills and knowledge. Most high maturity organizations have both training and formal mentoring programs. Training is tailored to the needs and experience of the students, with an emphasis on training in the application domain. High maturity organizations recognize the criticality of domain expertise; the organization may even be structured into product lines. Teams will usually have staffing requirements for expertise in the application domain as well as software engineering techniques and technologies.

A.3 Strategic Planning for Software Process Improvement

By now, readers may have appreciated the fact that "Software Process Improvement" does not occur overnight, and cannot be implemented on a "fad of the week" basis nor can the results be produced in "flash-in-the-pan" manner. If it is done that way then, it is doomed to failure. Implementing software process improvement involves the development of an overall strategy, requiring patience on the part of all parties involved: management, the developers, and the stakeholders. Many small steps are involved. If the development of the overall strategic plan is not performed judiciously, valuable time and money can be wasted.

The strategy for implementing process improvement should be addressed in a plan that accomplishes improvement actions as a series of small projects. The strategic plan should be one that is commensurate with the overall business objectives of the organization and should identify the resources (including personnel, capital outlays, and software and hardware tools) needed to execute the projects, the schedule, associated tasks, project responsibilities, and measures that will be utilized to indicate success or failure.

In developing a strategic plan, a team is organized consisting of the personnel who were trained and participated in the assessment, as well as any other personnel who have a vested interest in the outcome of the planning effort. Senior management also has an important role to perform.

As they say "there is no free lunch"! To expedite undertaking the various analyses that need to be performed and to achieve agreement on priorities, the planning process is typically conducted in a workshop mode, led by a facilitator. The approach found to most effectively lead to a sound process improvement strategy includes the following steps:

- Develop Process Improvement Proposals
- Evaluate the Proposals
- Rank the Proposed Projects
- Estimate the Implementation Schedule
- Obtain Management Commitment

In sum, we can say that successful process improvement planning is ensured when the planning process is based on key planning concepts:

- Strong linkage of processes to business objectives
- Involve the right people (People buy-in throughout)
- Using proven business planning methods.
- Facilitate productive communication and sharing of ideas among all participants (creating "process champions")
- An approach that does not lose sight of the big picture.

Author, based on her extensive experience in the Information Technology/Software Development industry (in the role of Quality Professional) finds that organizations adopting this approach have found that they can efficiently and rapidly move from assessment to implementing the needed changes with the confidence that their decisions are sound and will lead to a stronger software engineering capability.

A.4 Software CMM for Small Organizations

A frequently asked question is, "Can the Software CMM be used for small projects (or small organizations)?" However, since the definition of "small" is relative and sometimes challengingly ambiguous, we have shown this Table 3 as represented by the world famous SEI (Software Engineering Institute)

Table 3: Small Projects

Variant of "Small"	Team Size	Project Duration
Small	3-5	6 months
Very small	2-3	4 months
Tiny	1-2	2 months
Individual	1	1 week
Ridiculous!	1	1 hour

Note that small to tiny projects are in the range being addressed by Humphrey in his "Team Software Process" work [4] and the individual effort is in the range of the Personal Software Process [3]. TSP and PSP illustrate how CMM concepts are being applied to small projects. The "ridiculous" variant represents an interpretational problem.

One of the first challenges for small organizations in using the CMM is that their primary business objective is to survive! Even after deciding the status quo is unsatisfactory and process improvement will help, finding the resources and assigning responsibility for process improvement, and then following through by defining and deploying processes is a difficult business decision. The small organization tends to be victims of what is known as "Group Think Effect" and "Cognitive Dissonance"; i.e. the people in "small" organizations have a belief that: (Note that "Group Think Effect" is most often based on wrong beliefs that teams working in isolation have about themselves):

- We are all competent—people were hired to do the job, and we can't afford training in terms of either time or money
- We all communicate with one another—"osmosis" works because we're so "close"
- We are all heroes—we do whatever needs to be done, the rules don't apply to us (they just get in the way of getting the job done), we live with short cycle times and high stress

Even in "small" organizations/small projects, the following is required at a minimum:

- Documented customer (system) requirements
- Communication with customer (and end users)
- Agreed-to commitments
- Planning
- Documented processes
- Work breakdown structure

Some practices, however, deal with "large-project implementations." A small project is unlikely to need an SCM group (Software Configuration Management) or a Change Control Board... but configuration management and change control are always necessary; An independent SQA group may not be desirable, but objective verification that requirements are satisfied must be present. An independent testing group may not be established, but testing is always necessary. We thus see that it is only a question of degree of the formality involved but certainly not a complete absence of required processes!

The intent is critical even if the implementation is radically different between small organizations and large. Many of the large-project

ing the switch to the CMMI Product Suite is not only easier than it looks, but well worth it. For organizations already operating at a high Maturity Level, (recollect the discussion in the earlier section of this Appendix) the process of achieving CMMI is very straightforward. In fact, most organizations that have made a transition to the integrated CMM feel that CMMI validates the *best practices* already had in place due to requirements of the software CMM model. CMM-I, then, is just the matter of consolidating these practices institutionalizing them further.

CMMI enforces tying project objectives to organizational objectives, which is not only a good thing to do, but also a bad thing not to do. CMMI shows you exactly what you should be doing to improve your quality processes.

Although the CMMI models were developed in part to help larger organizations tackle complex issues across multiple disciplines, they can be tailored to meet the needs of smaller companies, especially if the companies apply the models purely for process improvement. Many smaller organizations aren't interested in using CMMI to benchmark themselves against other organizations. They don't need to commit to all the Process Areas and Practices. CMMI is designed so they can pick and choose among the *Process Areas* that have the most immediate applicability to them. The CMMI Product Suite is currently being applied across a range of organization sizes, from large to small.

A.6.2 Assessment Requirements for CMMI

First, let us understand what is "assessment". The Software Engineering Institute (SEI) defines this term as follows:

An examination of one or more processes by a trained team of professionals using an assessment reference model as the basis for determining strengths and weaknesses. An assessment is typically conducted in the context of process improvement or capability evaluation.

The Assessment Requirements for CMMI (Integrated Capability Maturity Model) comprise a set of high-level design criteria for developing, defining, and using assessment methods based on CMMI models. These requirements constitute an evolutionary progression from the CMM Appraisal Framework V1.0, which was produced originally to provide a common basis for assessment methods employing the Capability Maturity Model for Software.

There is what is known as "Assessment Classes"—each class is a family of assessment methods that satisfy a defined subset of requirements in the Assessment Requirements for CMMI (ARC). These classes are defined so as to align with typical usage modes of assessment. There are three classes of assessments for CMM-I: Class A, Class B and Class C. Depending on the class of assessment, the requirements vary (*Assessment Method Documentation, Plan and Preparation for the*

Assessment, Data Collection, Data Consolidation and Validation, Rating and Reporting Results)

Assessment teams use CMMI models as the basis for deriving the strengths and weaknesses of the processes investigated during an assessment. These findings, along with guidance provided by the practices in the model, are used to plan an improvement strategy for the organization.

The assessment principles for the CMMI Product Suite are the same as those for assessments using the Capability Maturity Model for Software:

- Start with an assessment reference model
- Use a formalized assessment process
- Focus the assessment on the sponsor's business goals
- Observe strict confidentiality and non-attribution of data
- Approach the assessment collaboratively
- Focus on follow-on process improvement activities

In connection with the CMM-I Assessments, various *roles and responsibilities* come into picture as defined by the SEI: (individual descriptions are beyond the scope of this appendix. The detailed description for each role is available at the SEI site www.sei.cmu.edu):

- The sponsor of the assessment
- The assessment team leader

The SEI has laid down a stringent set of requirements on the *Assessment Method Documentation*. Those descriptions too are available at the SEI site mentioned above.

REFERENCES

- [1] *Practices of High Maturity Organizations*—Mark C. Paulk Software Engineering Institute (1999 SEPG Conference, Atlanta, Georgia, 8–11 March 1999)
- [2] *Strategic Planning for Software Process Improvement: Techniques for Making the Hard Choices*—by Frank J. Koch and Emanuel R. Baker, Ph.D.
- [3] *Introduction to the Personal Software Process* by Watts S. Humphrey (Addison Wesley)
- [4] *Introduction to the Team Software Process* by Watts S. Humphrey (Addison-Wesley)
- [5] *The Personal Software Process: An Empirical Study of the Impact of PSP in Individual Engineers*—Will Hayes, James W. Over (Technical Report CMU/SEI-97-TR-001 ESC-TR-97-001 December 1997)

INDIAN SOFTWARE INDUSTRY IN PERSPECTIVE

B.1 Quality Certifications Scenario in India

According to Bill Curtis, Former Director of the Process Program, SEI, Author of the CMM (Capability Maturity Model):

"India is making progress on both the Software CMM and People CMM faster than any other nation. The pinnacle will be reached when Americans are seeking visas so they can learn and work in India"

According to N.R. Narayana Murthy, the CEO, Infosys Technologies:

For Quality Experts to successfully inspire everybody from top to bottom in an organization there are three basic requirements:

- *Quality has to be linked to business goals*
- *There has to be a sense of pride deeply instilled in your organization*
- *The third requisite for quality is a shared vision and commitment of top management*

With this backdrop, let us understand the Indian scenario on software practices maturity. Let us take a look at the statistics on the quality certifications scenario in India.

B.1.1 SW CMM Maturity Level 4 and 5 Organizations in India

The table on the next page presents some vital information in this context. It shows that there are quite a few Indian SW organizations that are at SW CMM maturity level 4 and 5. They are listed below in alphabetical order of their names.

Table 1

Organization	Maturity Level	Date of Appraisal
BFL Software Limited, Bangalore	4	June 1999
CG-Smith Software, Bangalore	5	Sept 1999
Citicorp Overseas Software Limited (COSL), Mumbai	4	Dec 1998
DCM Technologies, DCM ASIC Technology Limited, New Delhi	5	April 2000
DSQ Software, Chennai	4	June 1998
Future Software Private Limited, Chennai	4	June 1999
HCL Perot Systems, Noida and Bangalore	5	Feb 2000
Hughes Software Systems, Bangalore and Gurgaon	4	Jan 2000
IBM Global Services India, Bangalore	5	Nov 1999
i-flex solutions limited (formerly Citicorp Information Technology Industries Limited aka CITIL), Mumbai	4	Dec 1995
i-flex solutions limited (formerly Citicorp Information Technology Industries Limited aka CITIL) Data Warehouse Center of Excellence, Bangalore	5	Nov 1999
International Computers India Ltd (ICIL), Pune	5	Feb 1999
Motorola India Electronics Ltd. (MIEL), Bangalore	5	Nov 1993
Network Systems and Technologies (P) Ltd, Trivandrum	5	May 2000
Oracle Software India Limited, India Development Center, Bangalore	4	May 1999
Origin Information Technology (India) Ltd., Mumbai	4	May 1999
Origin Information Technology (India) Ltd., Mumbai	4	Oct 1999
Satyam Computer Services Ltd, India	5	March 1999
Silverline Technologies Limited, Mumbai, India	4	Dec 1999
Tata Consultancy Services, HP Centre, Chennai, India	5	July 1999
Tata Consultancy Services, SEEPZ, Mumbai, India	5	Aug 1999
Tata Consultancy Services, Shollinganallur, Chennai, India	5	Nov 1999
Tata Consultancy Services, US West, Chennai, India	5	April 1999
Tata Elxsi Limited, Bangalore, India	4	Aug 1999
Wipro GE Medical Systems, Bangalore, India	5	Jan 1999
Wipro Technologies, Enterprise Solutions Division, Bangalore, India	5	Dec 1998
Wipro Technologies, Global R & D (formerly Technology Solutions), Bangalore, India	5	June 1999

B.1.2 The P-CMM Scenario in India

People CMM is a process targeted at managing and developing an organization's work force and adopts the maturity framework of the Capability Maturity Model for software (CMM). The aim of PCMM is to radically improve the ability of software organizations to attract, develop, motivate, organize, and retain talent needed to continuously improve software development capability. PCMM consists of five

maturity levels that lay successive foundations for continuously improving talent, developing effective teams, and successfully managing the people assets of the organization. (In Chapter 9 we had provided an overview of the P-CMM model).

For a country like India, with its large assets in the form of skilled human resources, the relevance of People CMM needs no emphasis. A large number of Indian IT software and services companies have been quick to realize this and have either implemented or initiated programs. The chart below presents the P-CMM scenario in India.

Indian Software Industry (P-CMM Scenario)	
Company	Certified Level
Dattamatics Ltd.	Level 2
Datamatics Technologies Pvt. Ltd.	Level 2
Intelligroup Asia Pvt. Ltd.	Level 2
QAI (India) Limited	Level 2
MASTEK Ltd.	Level 3
Future Software Limited	Level 3
R S Software (India) Ltd.	Level 3
Siemens Information Systems Ltd.	Level 3
TechSpan India Ltd.	Level 3
Tata Consultancy Services	Level 4
Wipro Technologies	Level 5
IBM Global Services India Pvt. Ltd.	Level 5

B.1.3 List of Quality Certified Organizations in India

Provided below is the alphabetical list of quality certified organizations in India (both CMM, ISO, PCMM and others).

Table 2

Company Name	Quality Certifications
A	
A G Technologies Pvt Ltd.	• ISO 9001
Ace Software Exports Ltd	• ISO 9002
Aithent Technologies Pvt Ltd.	• SEI CMM Level 5
Ajuba Solutions (India) Pvt. Ltd.	• ISO 9001
Alcatel India Limited	• ISO 9001
ALSTOM Systems Limited	• ISO 9001
American Express (India) Pvt. Ltd.	• SEI CMM Level 2
AmitySoft Technologies Pvt Ltd	• ISO 9001
Aptech Limited.	• ISO 9001
Aviation Software Development Consultancy India Ltd.	• SEI CMM Level 3
AXA Business Services Pvt Ltd	• ISO 9002
Axes Technologies (I) Pvt Ltd	• ISO 9001

contd. ...

... contd.

Company Name	Quality Certifications
B	
BAeHAL Software Limited	• ISO 9001
Bangalore Software Services Ltd	• ISO 9001
Barry-Wehmiller International Resources Pvt Ltd	• ISO 9001
Bells Softech Limited	• ISO 9001
Beni Softwares Pvt Ltd	• ISO 9001
Bhari Information Technology Systems Pvt. Ltd.	• ISO 9001
Birla Technologies Ltd	• ISO 9001
Birlasoft Limited	• ISO 9001
Blue Star Infotech Ltd	• ISO 9001
BPL Telecom Ltd.	• SEI CMM Level 3
BT (Worldwide) Ltd.	• ISO 9001
C	
Canbank Computer Services Ltd.	• ISO 9001
CA-TCG Software Pvt. Ltd.	• ISO 9001
CDS International Ltd	• ISO 9001
Cellnext Solutions Ltd	• ISO 9001
Celstream Technologies Pvt Ltd	• ISO 9001
Cerebra Integrated Technologies Ltd	• ISO 9001
CGI Information Systems and Management Consultants Pvt Ltd.	• SEI CMM Level 4
CG-Smith Software Limited	• SEI CMM Level 5
CG-VAK Software & Exports Ltd.	• ISO 9001
Chandigarh Infotech Centre Ltd.	• ISO 9001
Churchill India Pvt. Ltd.	• ISO 9001
Cognizant Technology Solutions India Pvt. Ltd.	• PCMM Level 5
Compudyne Winfosystems Limited	• ISO 9001
Compulink Systems Pvt Ltd	• ISO 9001
Computer Clinic India Pvt. Ltd.	• ISO 9001
Computer Sciences Corporation India Pvt Ltd	• ISO 9001
Congruent Solutions Pvt. Ltd.	• ISO 9001
Consolidated Cybernetics Co. Pvt. Ltd.	• ISO 9001
Consulting Engineering Services (I) Ltd	• ISO 9001
Contech Software Ltd.	• SEI CMM Level 3
Convergent Software Limited	• SEI CMM Level 3
Covansys India (Private) Ltd.	• ISO 9001
Cressanda Solutions Limited	• ISO 9001
Crossword Software Pvt Ltd	• ISO 9001
D	
Datamatics Ltd.	• SEI CMM Level 5
Datamatics Technologies Pvt. Ltd.	• ISO 9001
DCM Datasystems Ltd.	• ISO 9001
DCM Technologies Limited	• SEI CMM Level 5
DDE ORG Systems Ltd.	• ISO 9001
Differential Technologies Limited	• ISO 9001
Digital GlobalSoft Ltd.	• SEI CMM Level 4
Divine India Limited	• ISO 9001
DPS Technologies India Pvt Ltd	• ISO 9002
DSL Software Ltd.	• ISO 9001
DSS Infotech International Ltd.	• ISO 9001

E

Eastern Software Systems Ltd.	• SEI CMM Level 5
Eclipse Systems Pvt. Ltd.	• ISO 9001
eFunds International Private Limited	• SEI CMM Level 3
Eonour Software Ltd.	• ISO 9001
Evolution InfoTech Limited	• ISO 9001

F

Financial Technologies (India) Ltd.	• ISO 9001
FirstApex	• ISO 9001
Future Software Limited	• PCMM Level 3

G

Gavs Information Service Pvt. Ltd.	• ISO 9001
Genisys Integrating Systems (I) Pvt. Ltd.	• ISO 9001
Geometric Software Solutions Company Ltd.	• SEI CMM Level 3
Global Business Technology Services Pvt Ltd	• ISO 9001
Global Dialnet Limited	• ISO 9001
Global Tech India Pvt. Ltd.	• ISO 9001
Globalsoft Pvt. Ltd.	• ISO 9001
Globsyn Technologies Ltd.	• ISO 9001
Godrej Infotech Ltd	• ISO 9002
Goldstone Technologies Ltd	• ISO 9001
Growth CompuSoft Exports Ltd.	• ISO 9002
GTIC (India) Pvt. Ltd	• ISO 9001
GTL Limited	• SEI CMM Level 4
Gurukulonline Learning Solutions (P) Ltd	• ISO 9001

H

HCL Infosystems Ltd	• ISO 9001
HCL Perot Systems Pvt Ltd	• ISO 9001
HCL Technologies Ltd	• SEI CMM Level 5
Hewlett-Packard India Limited	• SEI CMM Level 5
Hexaware Technologies Limited	• ISO 9001
Hinduja TMT Ltd	• ISO 9001
HMA STARware Ltd	• ISO 9001
Holool E-Business Pvt Ltd	• ISO 9001
Honeywell India Software Operations Pvt. Ltd.	• SEI CMM Level 4
HTC Software Development Centre	• SEI CMM Level 3

I

i2 Technologies India Pvt. Ltd.	• ISO 9001
IBILT Technologies Ltd	• ISO 9002
IBM Global Services India Pvt. Ltd.	• PCMM Level 5
IBS Software Services (P) Ltd	• SEI CMM Level 5
ICICI Infotech Services Ltd.	• SEI CMM Level 3
Ideaspace Solutions Ltd	• ISO 9001
i-flex solutions ltd.	• SEI CMM Level 5
Indchem Software Technologies Ltd	• SEI CMM Level 4
Indus Software Pvt. Ltd.	• ISO 9001
Information Technology Park Ltd.	• ISO 9000
Infosys Technologies Ltd.	• PCMM Level 5
Infotech Enterprises Ltd.	• ISO 9002

... contd.

Company Name	Quality Certifications
Infozech Software (P) Ltd.	• ISO 9001
InfraSoft Technologies Limited	• ISO 9001
Integra Software Services Private Limited	• ISO 9001
Intelligroup Asia Pvt. Ltd.	• PCMM Level 2
Intergraph Consulting Private Limited	• SEI CMM Level 5
iSeva Systems Pvt Ltd	• ISO 9001
IT Solutions (India) Pvt. Ltd.	• SEI CMM Level 5
ITC Infotech India Ltd	• SEI CMM Level 5
ITTI Limited	• ISO 9001
Ivega Corporation Pvt Ltd	• ISO 9001
IVL India Pvt. Ltd.	• ISO 9001
J	
JK Technosoft Ltd	• ISO 9001
JP Mobile (India) Limited	• ISO 9001
K	
Kale Consultants Ltd.	• ISO 9001
Kals Information Systems Ltd	• ISO 9001
Kanbay Software (India) Pvt. Ltd.	• ISO 9001
Karvy Consultants Ltd	• ISO 9002
Keane India Ltd	• ISO 9001
KLG Systel Ltd.	• ISO 9001
KPIT Cummins Infosystems Limited.	• SEI CMM Level 4
L	
Lambent Technologies Pvt Ltd	• ISO 9001
Lapiz Digital Services	• ISO 9001
Larsen & Toubro Infotech Limited	• ISO 9001
Laser Soft Infosystems Ltd	• ISO 9001
LG Soft India Pvt. Ltd.	• SEI CMM Level 5
Linc Software Services Pvt. Ltd.	• ISO 9001
Live Tech Solutions (P) Ltd	• ISO 9000
M	
Majoris Systems Pvt. Ltd.	• ISO 9001
MASCON Global Limited	• ISO 9000
Mascot Systems Ltd.	• ISO 9001
MASTEK Ltd.	• PCMM Level 3
Medicom Solutions (P) Ltd	• ISO 9001
Megasoft Limited	• ISO 9001
Melstar Information Technologies Ltd	• SEI CMM Level 3
Micro Technologies (India) Ltd	• ISO 9001
Momentum India Private Limited	• SEI CMM Level 4
Motorola India Electronics Private Ltd.	• SEI CMM Level 5
Mphasis BFL Ltd.	• ISO 9001
N	
NatureSoft Private Limited	• ISO 9001
Navayuga Infotech Pvt. Ltd.	• ISO 9001
NeilSoft Limited	• ISO 9001
Network Programs (India) Ltd.	• SEI CMM Level 3

Newgen Imaging Systems (P) Ltd.	• ISO 9001
NIIT Ltd.	• SEI CMM Level 5
NSE.IT Ltd.	• SEI CMM Level 3
Nucleus Software Exports Pvt. Ltd.	• SEI CMM Level 5

O

Ontrack Systems Limited	• ISO 9001
Onward Technologies Limited	• ISO 9002
Oracle India Private Limited.	• SEI CMM Level 4
Oracle Solution Services (India) Pvt Ltd	• SEI CMM Level 4
OrbiTech Solutions Limited	• SEI CMM Level 5

P

Paharpur Business Centre	• ISO 9002
Paragon Solutions (I) Pvt. Ltd.	• SEI CMM Level 3
Patni Computer Systems Ltd.	• ISO 9001
Pentamedia Graphics Limited	• ISO 9001
Pentasoftware Technologies Limited	• SEI CMM Level 4
Phoenix Global Solutions (India) Pvt. Ltd.	• SEI CMM Level 4
Pinnacle Info Solutions Pvt. Ltd.	• SEI CMM Level 4
Planet PCI Infotech Ltd	• ISO 9001
PlanetAsia.com Limited	• SEI CMM Level 4
Polaris Software Lab Ltd	• SEI CMM Level 5
Premier Technology Group Pvt. Ltd.	• ISO 9001
PSI Data Systems Ltd.	• ISO 9001

Q

QAI (India) Limited	• PCMM Level 2
Quinnnox Consultancy Services Ltd	• SEI CMM Level 5

R

R S Software (India) Ltd.	• PCMM Level 3
R Systems International Limited	• SEI CMM Level 4
Ram Informatics Ltd.	• ISO 9001
Ramco Systems Ltd	• SEI CMM Level 4
Rave Technologies (India) Pvt Ltd	• ISO 9001
RelQ Software Pvt Ltd	• ISO 9001
Rishabh Software Pvt Ltd	• ISO 9001
River Run Software Group	• ISO 9001
RMSI Private Limited	• ISO 9001
Robert BOSCH India Limited	• SEI CMM Level 3
Rolta India Ltd.	• ISO 9001

S

SAP India Pvt. Ltd.	• ISO 9002
Sasken Communication Technologies Limited	• SEI CMM Level 5
Satyam Computer Services Ltd.	• SEI CMM Level 5
Scicom Infotech Pvt. Ltd	• ISO 9001
Siemens Information Systems Ltd.	• SEI CMM Level 5
Siemens Public Communications Network Ltd.	• ISO 9001
Sierra Atlantic Software Services Limited	• ISO 9001
Siri Technologies Private Limited	• ISO 9001
Sitel India Pvt Ltd	• ISO 9000
Sobha Renaissance Information Technology	• SEI CMM Level 5
Softek Limited	• ISO 9001

... contd.

Company Name	Quality Certifications
SoftProjex (India) Limited	• ISO 9001
Solitar Systems (India) Pvt Ltd	• ISO 9001
SolutionNET India (Pvt) Ltd	• ISO 9001
Sonata Software Limited	• ISO 9001
Speck Systems Limited	• ISO 9001
SRA Systems Ltd.	• SEI CMM Level 4
Srishiti Software Private Limited	• SEI CMM Level 3
SSI Technologies	• ISO 9002
Subex Systems Limited	• ISO 9001
Summit Information Technologies Ltd	• ISO 9001
Sundaram Infotech Solutions	• ISO 9001
Syntel (India) Ltd.	• SEI CMM Level 5
T	
Tasaa Netcom Private Limited	• ISO 9001
Tata Consultancy Services	• PCMM Level 4
Tata Elxsi Ltd.	• SEI CMM Level 5
Tata Infotech Ltd.	• ISO 9002
Tata Interactive Systems	• ISO 9001
Tata Technologies Limited	• ISO 9001
TCG Software Services Pvt. Ltd.	• ISO 9001
TCIL BellSouth Ltd.	• ISO 9001
TechSpan India Ltd.	• SEI CMM Level 3
Tektronix Engineering Development I) Ltd.	• ISO 9000
Telecommunications Consultants India Ltd.	• ISO 9001
Temenos India Pvt. Ltd.	• ISO 9000
Thinksoft Global Services (P) Ltd	• ISO 9001
Three S Solutions Ltd	• ISO 9002
U	
UshaComm India Pvt. Ltd.	• ISO 9001
Ushus Technologies Pvt Ltd	• ISO 9001
V	
ValueMomentum Software Services Pvt Ltd	• ISO 9001
Vision Healthsource (I) Pvt. Ltd.	• ISO 9001
Vispark Solutions (I) Pvt Ltd	• ISO 9001
VJIL Consulting Limited	• ISO 9001
VXL eTech Ltd	• SEI CMM Level 3
W	
Web Spiders (I) Pvt Ltd	• ISO 9001
Webify Services (India) Private Limited	• ISO 9001
Wipro Technologies	• SEI CMM Level 5
WNS Global Services	• ISO 9001
X	
Xansa (India) Ltd	• SEI CMM Level 5
Xcelvision Technologies Limited	• ISO 9001
Xerox Modicorp Limited	• SEI CMM Level 3
Z	
Zensar Technologies Limited	• EFQM

B.1.3 Top 20 software companies in India

Below is the NASCOM's list of Top 20 software companies (Software and Services Exports)

Table 3: Top 20 IT Software and Services Exporters from India

Rank	Company	Rs. crore	US\$ million
1.	Tata Consultancy Services	3,882	813
2.	Infosys Technologies Limited	2,553	535
3.	Wipro Technologies	2,256	481
4.	Satyam Computer Services Limited	1,703	357
5.	HCL Technologies. Ltd.	1,319	277
6.	IBM Global Services India Pvt. Ltd.	764	160
7.	Patni Computer Systems	732	153
8.	Silverline Technologies	603	126
9.	Mahindra-British Telecom Limited	541	113
10.	Pentsoft Technologies Ltd.	459	96
11.	HCL Perot Systems Limited	449	94
12.	Pentamedia Graphics Ltd.	431	90
13.	NIIT Limited	400	84
14.	Mascot Systems Ltd.	399	84
15.	i-Flex Solutions Ltd.	392	82
16.	Digital Globalsoft Ltd.	331	69
17.	Mphasis BFL Group (Consolidated)	313	66
18.	Mascon Global Limited	307	64
19.	Orbitech	264	55
20.	Mastek Limited	259	54

Note: Companies are ranked on the basis of export revenues from India for Indian and MNC Companies

Source: NASCOM

Table 4: Indian IT Software and Services: Export Revenues Ranking (2001-02)

Rank	Company	Rs. crore	US\$ million
1.	Tata Consultancy Services	3,882	813
2.	Infosys Technologies Ltd.	2,553	535
3.	Wipro Ltd.	2,298	481
4.	Satyam Computer Services Ltd.	1,703	357
5.	HCL Technologies Ltd.	1,320	277
6.	Patni Computer Systems Ltd.	732	153
7.	Silverline Technologies Ltd.	603	126
8.	Mahindra British Telecom Ltd.	541	113
9.	Pentsoft Technologies Ltd.	460	96
10.	HCL Perot Systems Ltd.	449	94
11.	Mascot Systems Ltd.	403	84
12.	NIIT	400	84
13.	i-Flex Solutions Ltd.	392	82
14.	Mphasis BFL	313	66
15.	Mascon Global Ltd.	307	64
16.	Mastek Ltd.	259	54
17.	Birlasoft Ltd.	259	54
18.	Polaris Software Lab Ltd.	247	52
19.	L&T Information Technology Ltd.	245	51
20.	Hexaware Technologies Ltd.	241	50

Note: Does not include MNC companies

Source: NASCOM

Table 5

Top 20: Software exporters (MNCs) 2001-02

		(no. of employees)
1.	IBM Global Services India Ltd.	3,100
2.	Cognizant Technology Solutions	2,712
3.	Oracle India Pvt. Ltd.	2,000
4.	Hughes Software Systems Ltd.	1,500
5.	Hewlett-Packard (I) Software Operation Ltd.	1,489
6.	Digital Globalsoft	1,480
7.	Syntel	1,464
8.	Covansys (I) Ltd.	1,449
9.	PwC	1,200
10.	OrbiTech Solutions	1,191
11.	Siemens Information Systems Ltd.	1,187
12.	Xansa (India) Ltd.	892
13.	Motorola	800
14.	ST Microelectronics	800
15.	Texas Instruments India Pvt. Ltd	741
16.	Intel Tech India Pvt. Ltd.	700
17.	i2 Technologies	700
18.	Cisco	670
19.	Robert Bosch	629
20.	Huawei	500

(no. of employees as of March 2002)

Source: NASSCOM

B.2 The Indian Software Scenario in General

Global Scenario for Software Market is characterized by the following:

- Total value of software is estimated to be around \$600 billion+
- About half is for general-purpose or shrink-wrapped software products
- Other half is the software services market

Given this scenario, services segment is well suited for offshore work (low risk, low capital, manpower intensive....) in which low dependence on physical infrastructure is considered ok. This makes the scheme of things well suited for country like India as the software industry is mostly in service sector.

Opening of the economy has given spurt to the Indian Software Industry, which is young.

Earlier, there were only few players, with only one big player (the TCS). Later, economic liberalization took place in 90s. As a result, computer import became easy and cheaper, tax benefits and other facilities were declared for exporting, controls and regulations were eased, making export easy.

As seen in the tables above, today, there are more than 1000 software companies of which at least 30 are with more than 1000 employees. Major Indian IT players have a global presence. Many multinationals have centers in India—IBM, Microsoft, Cisco, GE, Oracle, Novell, Sun, Hughes etc.

B.3 Cost and Quality Advantage: Indian Software Industry

In the past few years, the Indian IT industry has pursued the goal of attaining the highest international standards of quality. A World Bank funded study conducted as early as 1992 to discuss Indian software strategies had concluded that more and more vendors in the US prefer to get their software developed undertaken in India for its quality and cost advantage.

Indian players have created a strong value proposition in the IT software and services arena. India enjoys advantages of people sophistication in terms of a very large pool of English speaking scientific personnel, varied and extensive skill sets in terms of technology, and offering services at globally competitive costs. India also boasts vendor sophistication—with more than 200 companies being quality accredited and serving the needs of over 255 Fortune 500 companies. Today, the world looks towards the Indian IT software and services industry for its good quality and high price performance. This is a matter of national pride for us.

According to McKinsey & Co., India has and will continue to have a growing number of vendors successfully working on complex projects across all areas of software and services, and performing at levels comparable to those of leading global players.

In March 2002, NASSCOM conducted a survey to ascertain the adoption of international quality standards by IT software and services companies in India. An analysis of top 300 companies in India revealed some interesting statistics. NASSCOM's Survey presents the following picture on organization's quality plans.

Quality Certification	No. of Companies
Already acquired ISO 9000 or SEI and other certification	216
Companies acquired quality certification by March 2002	6
Certification expected between April-December 2002	64
Additional certification expected during 2002	70
No plans at present	22

SEI Quality Assessment	No. of Companies as on 31st May, 2002	No. of Companies by 31st March, 2003
SEI CMMi	3	5
SEI CMM Level 5	42	46
SEI CMM Level 4	22	38
SEI CMM Level 3	24	34
SEI CMM Level 2	3	16
PCMM Level 5	2	2
PCMM Level 4	1	2
PCMM Level 3	5	6
PCMM Level 2	4	12

Note: Some companies have multiple certifications.

As of 27 June 2002, India had 85 companies at SEI CMM Level 5 assessment. It is more noteworthy that worldwide, only 42 organizations have acquired such assessment. The quality maturity of the Indian software industry can be measured from the fact that already 316 Indian software companies have acquired quality certifications and more companies are in pipeline to do so.

The other heartening feature has been the growing acceptance and adoption of the newly emerging *People-Capability Maturity Model* (People-CMM) by the Indian software industry. For a country like India, with its large assets in the form of skilled human resources, the relevance of People CMM needs no emphasis. A large number of Indian IT software and services companies have been quick to realize this and have either implemented or initiated programs. (In the previous section, we have presented the statistics on the P-CMM scene in India)

During 2001-02, three Indian companies achieved notable distinctions in CMMi certification. Polaris Software Lab was assessed to SEI CMMi Level 5 (Version 1.02) in October 2001, making it the first company in the world to acquire this quality standard. Recently, both Wipro and Infosys have also been assessed to CMMi Level 5 quality standards.

B.4 th Indian Software Industry: Problems & Challenges

Indian software exports are steadily growing. The largest number of quality certified software organizations are in India as seen in the previous section. However, although all this may appear to present a rosy picture, it is not that the Indian Software Industry is without any problems and challenges. In this section, we have provided a discussion on this point. The major problems and challenges seem to be:

- The Uneven Profile of the Indian Software Industry
- People Management Problems
- Infrastructure related problem
- Growing competition from other countries (especially China)

One of the major problems faced by the Indian software industry is its uneven profile. This is the point of discussion in this section. Indian software exports have shown very high growth rates for many years. Yet, behind this success lie a number of skews.

B.4.1 The Uneven Profile of Indian Software Exports

By almost any standards, the growth of India's software exports has been phenomenal. Exports began in 1974 but made limited impact until the 1980s. From that time on, growth rates have been consistently high, as the table below illustrates.

Table 6: Indian Software Exports and Growth Rates (1980-1997/98)

Year	Software Exports (US\$million)	Growth Rate
1980	4.0	
1981	6.8	70%
1982	13.5	99%
1983	18.2	35%
1984	25.3	39%
1985	27.7	9%
1986	38.9	40%
1987	54.1	39%
1988/89 (Apr-Mar)	69.7	29%
1989/90	105.4	51%
1990/91	131.2	24%
1991/92	173.9	33%
1992/93	219.8	26%
1993/94	314.0	43%
1994/95	480.9	53%
1995/96	668.0	39%
1996/97	997.0	49%
1997/98	1650.0	65%

Source: Dataquest (1998)

However, all is not quite what it seems. For example, as noted, these 'headline' figures represent gross foreign exchange earnings. Net earnings (less outflow of foreign exchange from India to pay for travel and living allowances of Indian software workers who undertake their contracts overseas, marketing, multinational profit repatriation, importation of hardware and software, etc.) are now estimated to be around 40% of the gross figures. The Indian software industry has also been characterized by an uneven profile along several dimensions. These skews have important implications for both the growth and earnings potential of Indian software, as described below.

1. Uneven Output: Services not Packages

Indian software exports have been dominated by export of software services, in the form of custom software work, rather than export of software